เอกสารประกอบการสอน

วิชา โครงสรางระบบคอมพิวเตอร์

(Computer System Organization)

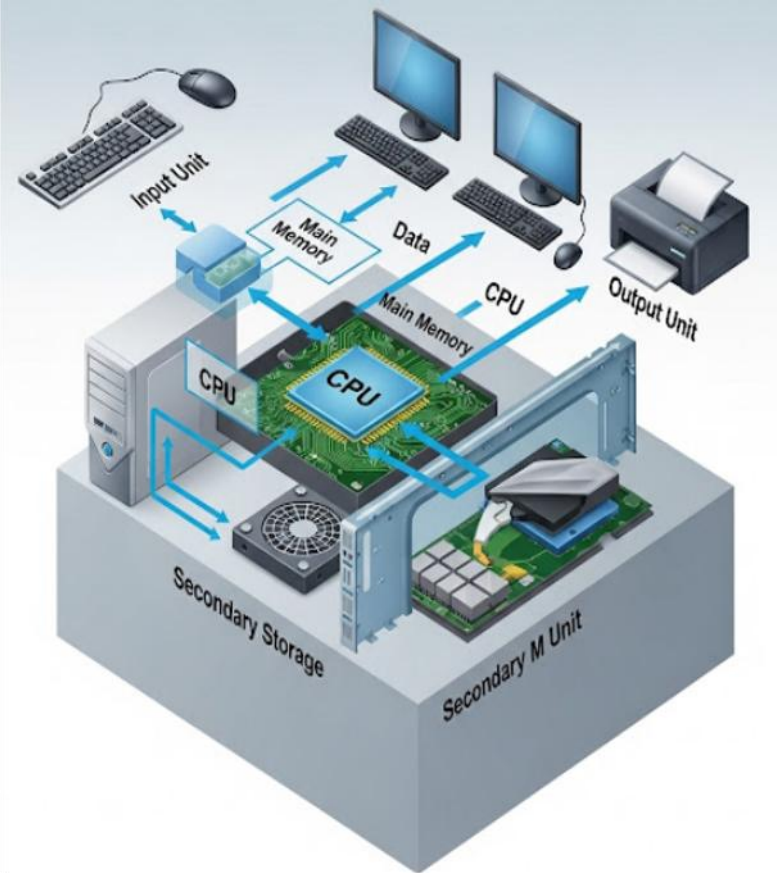
บทที่ 1 เรื่องโครงสร้างระบบคอมพิวเตอร์

(COMPUTER SYSTEM ORGANIZATION)

วัตถุประสงค์การเรียนรู้ (Learning Objectives)

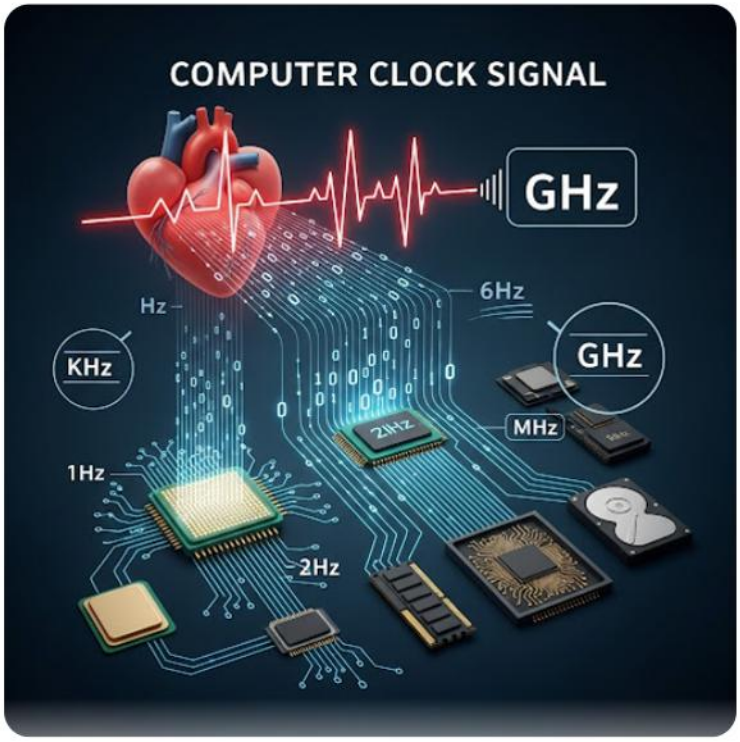
1. บอกชื่อและองค์ประกอบคอมพิวเตอร์ได้
2. อธิบายหน้าที่และหลักการสร้างสัญญาณนาฬิกา (Clock) ของคอมพิวเตอร์ได้
3. อธิบายการนำสัญญาณนาฬิกกา (Clock) ไปใช้ในส่วนต่างๆ ของคอมพิวเตอร์ได้
4. อธิบายหน่วยวัดความเร็วของคอมพิวเตอร์ (Hertz, KHz, MHz, GHz) ได้อย่างถูกต้อง
5. อธิบายการนำสัญญาณนาฬิกกา (Clock) ที่ส่งไปให้อุปกรณ์แต่ละส่วนมีความเร็วไม่เท่ากันได้
6. อธิบายวิธีการคำนวนเพื่อเปรียบเทียบ ประสิทธิภาพความเร็ว (Speed up) ของ CPU ในสถานการณ์ต่างๆ ได้
7. อธิบายวิวัฒนาการของคอมพิวเตอร์ตั้งแต่ยุคที่ 1 ถึงยุคที่ 5 รวมถึงเทคโนโลยีหลักที่ใช้ในแต่ละยุคได้
8. อธิบายโครงสร้างและหน้าที่ของหน่วยประมวลผลกลาง (CPU Structure and Function) ได้
9. อธิบายการทำงานของวงรอบคำสั่ง (Instruction Cycle) ทั้ง Fetch และ Execute ได้อย่างถูกต้อง
10. อธิบายส่วนประกอบของรูปแบบคำสั่ง (Instruction Format) ที่ใช้ในการประมวลผลข้อมูลได้
11. อธิบายหลักการเชื่อมต่อระบบบัสคอมพิวเตอร์ได้ (System Interconnection - BUS)
12. อธิบายการทำงานของหน่วยรับและส่งข้อมูล (Input/Output Unit) รวมถึงความสำคัญของพอร์ตต่างๆ ได้
13. บอกหน้าที่ของหน่วยความจำ (Memory Unit) ในระบบคอมพิวเตอร์ได้
14. อธิบายลำดับชั้นของหน่วยความจำ (Memory Hierarchy) ในระบบคอมพิวเตอร์ได้
15. อธิบายหลักการทำงานของหน่วยความจำสำรองแบบต่างๆ ได้
16. อธิบายหลักการทำงานของหน่วยความจำแคช (Cache Memory) เมื่อมีการทำงานแนวคิด Locality of Reference ได้
17. อธิบายวิธีการคำนวณและสัดส่วนการเข้าถึงแคช (Hit Ratio) ได้
18. อธิบายการเชื่อมโยงข้อมูลของหน่วยความจำแคช (Cache Mapping) แบบ Direct Mapping, Fully Associative Mapping, Set Associative Mapping ได้อย่างถูกต้อง
19. อธิบายการคำนวณจำนวนบิต สำหรับ Tag, Index/Set, และ Offset ในการออกแบบหน่วยความจำแคชด้วยวิธีการ Mapping แบบต่างๆ ได้
20. อธิบายอัลกอริทึมการเปลี่ยนบล็อกในแคชแบบต่างๆ ได้
21. อธิบายข้อดีและข้อเสียของนโยบายการเขียนข้อมูล (Write Policy) ในแคช ทั้งแบบ Write Through และ แบบWrite Back ได้

1. บทนำและแนวคิดพื้นฐานเกี่ยวกับคอมพิวเตอร์ (Introduction & Basic Concepts)



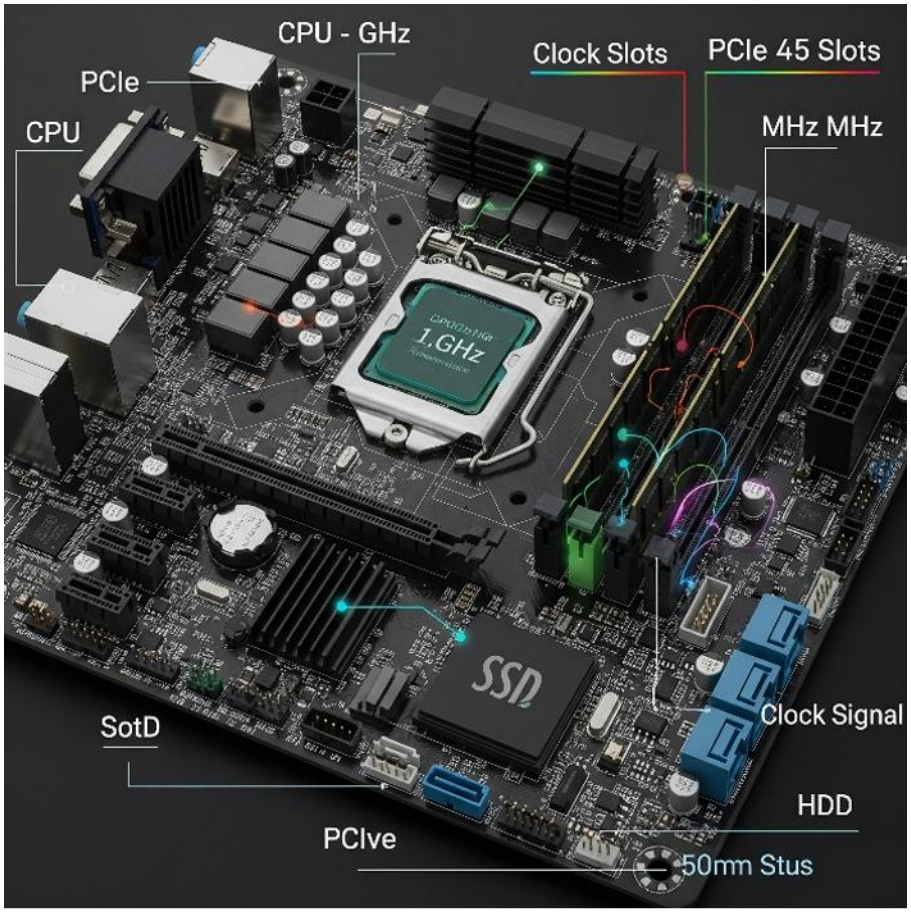
รูปที่ 1-1 แผนผังองค์ประกอบและการทำงานของระบบคอมพิวเตอร์

คอมพิวเตอร์คืออะไร : คอมพิวเตอร์ไม่ได้เป็นเพียงเครื่องมือสำหรับงานเฉพาะทางเท่านั้น แต่เป็น **เครื่องจักรอิเล็กทรอนิกส์อเนกประสงค์** ที่มีความสามารถในการประมวลผลข้อมูลตามชุดคำสั่ง (โปรแกรม) ที่กำหนดไว้ล่วงหน้าอย่างรวดเร็วและแม่นยำ องค์ประกอบหลักๆ ได้แก่ หน่วยรับข้อมูล (Input Unit), หน่วยประมวลผลกลาง (CPU), หน่วยความจำหลัก (Main Memory), หน่วยแสดงผล (Output Unit), และหน่วยความจำสำรอง (Secondary Storage) ซึ่งแต่ละส่วนทำงานร่วมกันเป็นระบบเพื่อให้คอมพิวเตอร์สามารถรับข้อมูล ประมวลผล จัดเก็บ และแสดงผลลัพธ์ได้ตามต้องการ



รูปที่ 1-2 ความเร็วสัญญาณนาฬิกา (Clock Speed) ในหน่วยต่าง ๆ

สัญญาณนาฬิกา (Clock) และความเร็วของคอมพิวเตอร์: สัญญาณนาฬิกาเปรียบเสมือนจังหวะการเต้นของหัวใจคอมพิวเตอร์ เป็นสัญญาณไฟฟ้าที่สลับกันระหว่าง 0 และ 1 อย่างสม่ำเสมอในอัตราที่คงที่ เพื่อใช้เป็นตัวควบคุมการทำงานพร้อมกันของวงจรดิจิทัลต่างๆ ภายใน CPU และส่วนประกอบอื่นๆ เช่นการดึงคำสั่ง การคำนวณ การถ่ายโอนข้อมูล **หน่วยเฮิรตซ์ (Hertz)** (Hz) คือหน่วยวัดความถี่ของสัญญาณนาฬิกา ซึ่งบอกจำนวนรอบสัญญาณต่อวินาที (1 Hz = 1 รอบ/วินาที) โดยคอมพิวเตอร์ในปัจจุบันมี



รูปที่ 1-3 ส่วนประกอบเมนบอร์ดและการจัดวางเลย์เอาต์ (Motherboard Layout)

ความเร็วสูงมาก จึงใช้หน่วยเป็น KHz (103 Hz), MHz (106 Hz), และ GHz (109 Hz) ซึ่งหมายถึงจำนวนพันล้านรอบต่อวินาที ตัวอย่างเช่น CPU 3.5 GHz หมายถึง CPU ทำงาน 3.5 พันล้านรอบสัญญาณต่อวินาที

การสร้างสัญญาณนาฬิกา

สัญญาณนาฬิกาในคอมพิวเตอร์จะมาจาก **ตัวกำเนิดสัญญาณนาฬิกา (Clock Generator)** ซึ่งมักจะเป็นวงจรออสซิลเลเตอร์ (Oscillator) ที่ใช้ผลึกควอตซ์ (Quartz Crystal) เป็นส่วนประกอบหลัก ผลึกควอตซ์มีคุณสมบัติในการสั่นสะเทือนด้วยความถี่ที่แม่นยำและคงที่เมื่อได้รับกระแสไฟฟ้า วงจรนี้จะสร้างสัญญาณพัลส์ (Pulse) ที่เป็นจังหวะขึ้นลงสลับกันระหว่าง 0 และ 1 ซึ่งเรียกว่าสัญญาณนาฬิกา หรือ Clock Signal

1.2 การนำสัญญาณนาฬิกา (Clock) ไปใช้ในส่วนต่างๆ ของคอมพิวเตอร์

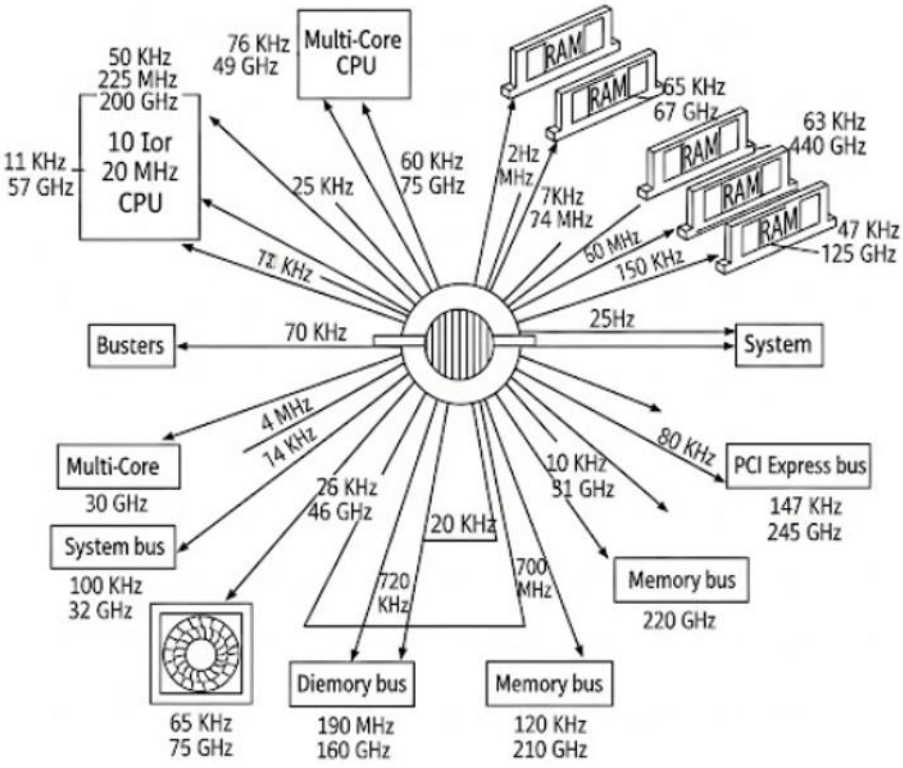
สัญญาณนาฬิกาที่สร้างขึ้นจะถูกส่งไปยังส่วนประกอบต่างๆ ของคอมพิวเตอร์เพื่อ **ซิงโครไนซ์ (Synchronize)** การทำงานของอุปกรณ์เหล่านั้นเป็นตัวกำหนดจังหวะการทำงานให้ทุกส่วนทำงานพร้อมกัน

- **หน่วยประมวลผลกลาง (CPU):** สัญญาณนาฬิกาเป็นหัวใจหลักในการควบคุมการทำงานของ CPU ทุกคำสั่งที่ CPU ประมวลผล เช่น การดึงคำสั่ง (Fetch), การตีความคำสั่ง (Decode), การประมวลผล (Execute) และการจัดเก็บผลลัพธ์ (Write-back) จะเกิดขึ้นตามจังหวะของสัญญาณนาฬิกา ความเร็วของ CPU มักถูกวัดเป็น Hertz (Hz) เช่น MHz หรือ GHz ซึ่งบ่งบอกถึงจำนวนรอบสัญญาณนาฬิกาต่อวินาที ยิ่งค่า Hertz สูงเท่าไหร่ CPU ก็ยิ่งทำงานได้เร็วขึ้นเท่านั้น
    - 1 KHz หมายถึง 1,000 รอบสัญญาณนาฬิกาต่อวินาที
    - 1 MHz หมายถึง 1,000,000 รอบสัญญาณนาฬิกาต่อวินาที
    - 1 GHz หมายถึง 1,000,000,000 รอบสัญญาณนาฬิกาต่อวินาที

- **หน่วยความจำ (Memory):** การอ่านและเขียนข้อมูลลงในหน่วยความจำหลัก (RAM) ก็ถูกควบคุมด้วยสัญญาณนาฬิกา เพื่อให้มั่นใจว่า CPU และหน่วยความจำทำงานประสานกันได้อย่างถูกต้อง การเข้าถึงข้อมูลในหน่วยความจำจะต้องใช้เวลาที่สัมพันธ์กับจังหวะของสัญญาณนาฬิกา
- **อุปกรณ์รับส่งข้อมูล (Input/Output Devices):** อุปกรณ์ I/O ต่างๆ เช่น ฮาร์ดดิสก์, คีย์บอร์ด, เมาส์, พรินเตอร์ ก็มีการทำงานที่ต้องอาศัยสัญญาณนาฬิกาในการควบคุมจังหวะการรับส่งข้อมูล ถึงแม้ว่าความเร็วในการทำงานของอุปกรณ์เหล่านี้จะแตกต่างจาก CPU แต่ก็ยังคงต้องมีสัญญาณนาฬิกาเพื่อควบคุมกลไกภายในของอุปกรณ์นั้นๆ

1.3 การนำสัญญาณนาฬิกกา (Clock) ที่ส่งไปให้อุปกรณ์แต่ละส่วนมีความเร็วไม่เท่ากัน

แม้ว่าจะมีสัญญาณนาฬิกาหลักที่มาจาก Clock Generator แต่ความเร็วของสัญญาณนาฬิกาที่ส่งไปให้อุปกรณ์แต่ละส่วนอาจไม่เท่ากันด้วยเหตุผลดังนี้:

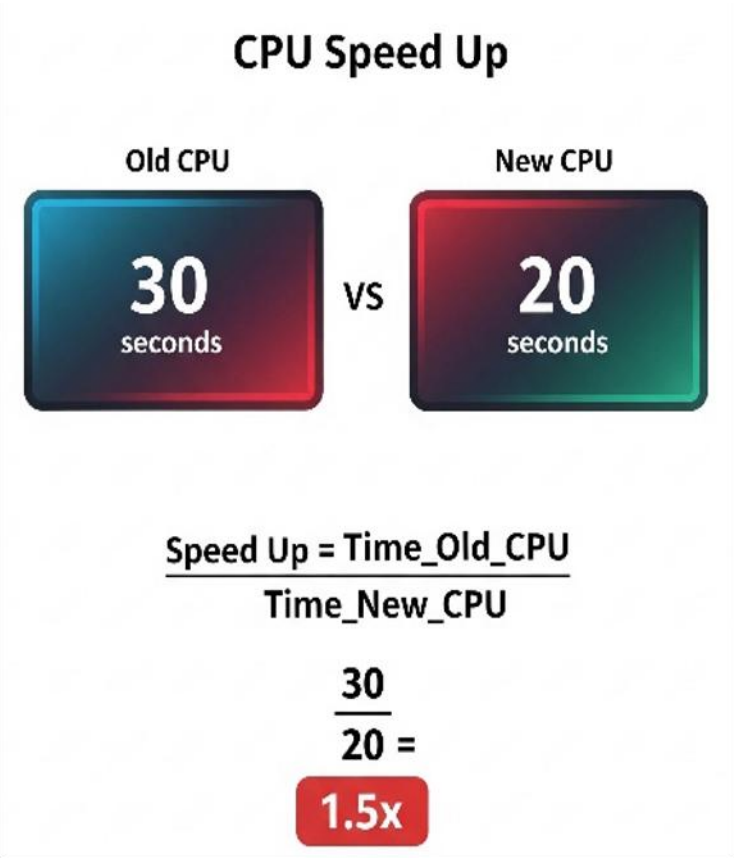


รูปที่ 1-4 การทำงานของเครื่องกำเนิดสัญญาณนาฬิกาคอมพิวเตอร์และความถี่ของระบบบัส (System Bus Frequencies)

1. **ความสามารถของอุปกรณ์ (Device Capabilities):** อุปกรณ์แต่ละชนิดมีความสามารถในการประมวลผลและทำงานที่ความเร็วที่แตกต่างกันไป CPU ต้องการความเร็วที่สูงมากเพื่อประมวลผลคำสั่งจำนวนมากในเวลาอันสั้น ในขณะที่อุปกรณ์ I/O บางชนิด เช่น คีย์บอร์ด หรือเมาส์ ไม่จำเป็นต้องใช้ความเร็วสูงเท่า CPU เพราะลักษณะการทำงานไม่ได้ต้องการการซิงโครไนซ์ที่ละเอียดซับซ้อนขนาดนั้น การป้อนสัญญาณนาฬิกาที่เร็วเกินไปให้กับอุปกรณ์ที่ทำงานได้ช้ากว่าอาจทำให้เกิดข้อผิดพลาดหรือความเสียหายได้

2. **การประหยัดพลังงานและการจัดการความร้อน (Power Saving and Heat Management):** อุปกรณ์ที่ทำงานด้วยความเร็วสูงใช้พลังงานมากกว่าและสร้างความร้อนได้มากกว่า การควบคุมความเร็วสัญญาณนาฬิกาให้เหมาะสมกับความจำเป็นของแต่ละอุปกรณ์จะช่วยประหยัดพลังงานและลดปัญหาความร้อนสะสมในระบบ
3. **การทำงานของระบบบัส (Bus Speed):** การส่งข้อมูลระหว่าง CPU, หน่วยความจำ และอุปกรณ์ I/O จะผ่านระบบบัส (Bus) ซึ่งมีความเร็วในการส่งข้อมูลที่จำกัด บัสแต่ละประเภทอาจมีสัญญาณนาฬิกาเป็นของตัวเอง หรือใช้สัญญาณนาฬิกาหลักแต่มีการหาร (Divide) ความถี่ลงมาเพื่อให้เข้ากับความเร็วของบัสและอุปกรณ์ที่เชื่อมต่ออยู่ ตัวอย่างเช่น Front Side Bus (FSB) หรือ System Bus มีความเร็วที่สัมพันธ์กับความเร็วของ CPU แต่ก็มีส่วนประกอบอื่นๆ เช่น PCI Express (PCIe) หรือ USB ที่มีมาตรฐานความเร็วของตัวเอง
4. **ความเสถียรของระบบ (System Stability):** การออกแบบระบบคอมพิวเตอร์ต้องคำนึงถึงความเสถียรโดยรวม การให้สัญญาณนาฬิกาที่เร็วเกินไปแก่อุปกรณ์ที่ไม่รองรับ หรือการทำงานที่ความเร็วไม่สอดคล้องกัน อาจทำให้เกิดปัญหาความไม่เสถียรของระบบหรือเกิดข้อผิดพลาดในการทำงานได้

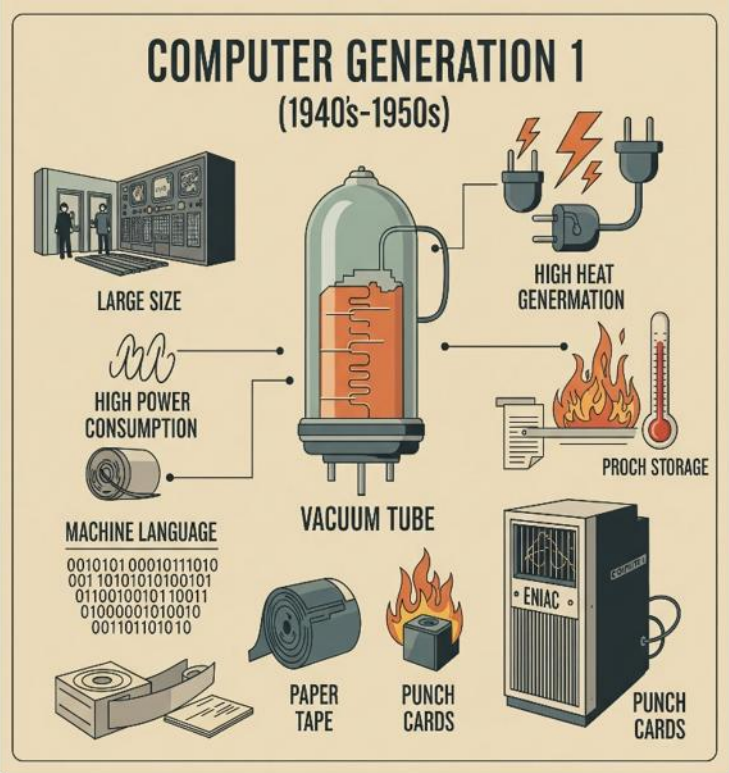
ดังนั้น จึงมีการใช้ตัวแบ่งความถี่ (Frequency Divider) หรือวงจร PLL (Phase-Locked Loop) เพื่อปรับความเร็วของสัญญาณนาฬิกาจาก Clock Generator ให้เหมาะสมกับความต้องการของแต่ละอุปกรณ์ เพื่อให้ระบบคอมพิวเตอร์สามารถทำงานได้อย่างมีประสิทธิภาพและเสถียรภาพสูงสุด



รูปที่ 1-5 การคำนวณเปรียบเทียบประสิทธิภาพความเร็วของซีพียู

การคำนวณค่าความเร็ว (Speed up): การเปรียบเทียบประสิทธิภาพของ CPU สองตัว เราสามารถคำนวณค่า Speed up ได้จากอัตราส่วนของเวลาที่ใช้ในการทำงานเดิมด้วย CPU ตัวเก่า หารด้วยเวลาที่ใช้กับ CPU ตัวใหม่ เช่น ถ้าโปรแกรมใช้เวลา 30 วินาทีบน CPU เก่า และ 20 วินาทีบน CPU ใหม่ ค่า Speed up จะเป็น 30/20 = 1.5 เท่า นั่นคือ CPU ใหม่ทำงานเร็วกว่า 1.5 เท่า

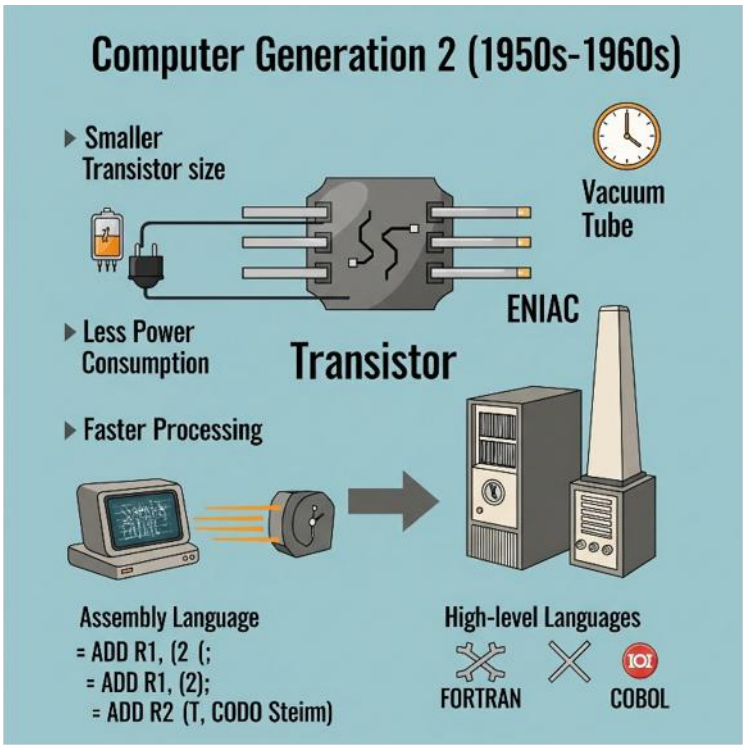
2. วิวัฒนาการของคอมพิวเตอร์ (Evolution of Computers)



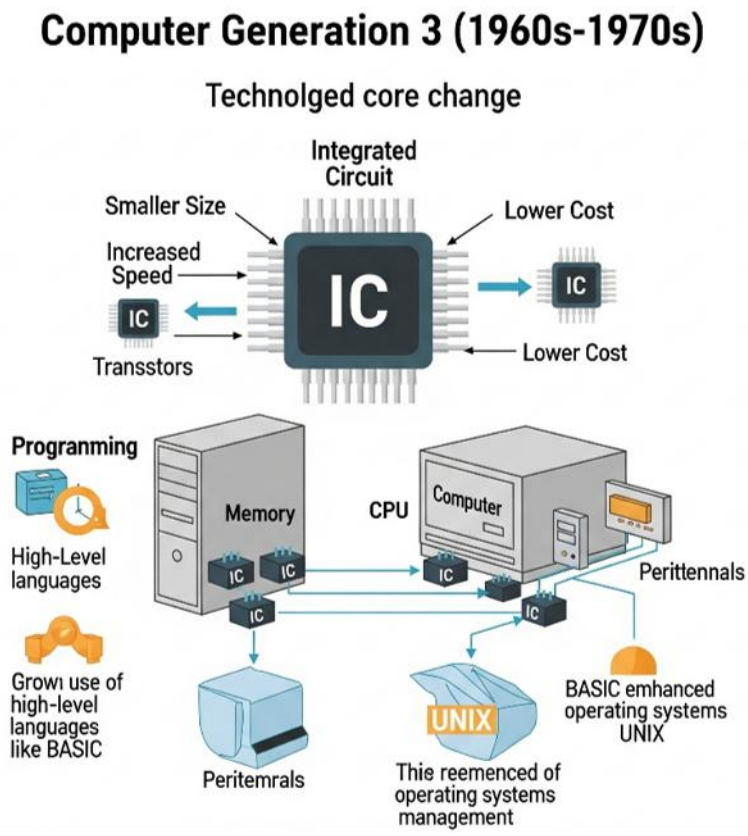
รูปที่ 1-6 ลักษณะเฉพาะและเทคโนโลยีคอมพิวเตอร์ในยุคที่ 1

ยุคที่ 1 (ทศวรรษ 1940-1950s): ยุคนี้เป็นจุดเริ่มต้นของคอมพิวเตอร์ดิจิทัล โดยมี **หลอดสูญญากาศ (Vacuum Tube)** เป็นหัวใจหลักในการประมวลผล หลอดเหล่านี้มีขนาดใหญ่ กินไฟมาก ผลิตความร้อนสูง ทำให้คอมพิวเตอร์ในยุคนี้มีขนาดเท่าห้องใหญ่ๆ และทำงานช้ามาก ตัวอย่างเช่น ENIAC (Electronic Numerical Integrator and Computer) ซึ่งเป็นคอมพิวเตอร์ดิจิทัลอิเล็กทรอนิกส์เครื่องแรกๆ การเขียนโปรแกรมยังคงต้องใช้ **ภาษาเครื่อง (Machine Language)** ซึ่งเป็นรหัสไบนารี (0s และ 1s) โดยตรง ทำให้การเขียนโปรแกรมเป็นเรื่องที่ยากและซับซ้อนมาก และการบันทึกข้อมูลใช้เทปกระดาษหรือบัตรเจาะรู

ยุคที่ 2 (ทศวรรษ 1950-1960s): การปฏิวัติครั้งสำคัญเกิดขึ้นเมื่อ **ทรานซิสเตอร์ (Transistor)** เข้ามาแทนที่หลอดสูญญากาศ ทรานซิสเตอร์มีขนาดเล็กกว่ามาก กินไฟน้อยกว่า และทำงานได้เร็วกว่าหลอดสูญญากาศอย่างเห็นได้ชัด ทำให้คอมพิวเตอร์มีขนาดเล็กลง ประหยัดพลังงานขึ้น และมีประสิทธิภาพสูงขึ้นอย่างก้าวกระโดด ในยุคนี้เริ่มมีการพัฒนา **ภาษาแอสเซมบลี (Assembly Language)** ซึ่งเป็นภาษาระดับต่ำที่ใกล้เคียงกับภาษาเครื่องแต่ใช้สัญลักษณ์ที่เข้าใจง่ายขึ้น และเริ่มมี **ภาษาโปรแกรมระดับสูง (High-level Languages)** เช่น FORTRAN และ COBOL ทำให้การเขียนโปรแกรมง่ายขึ้นมาก



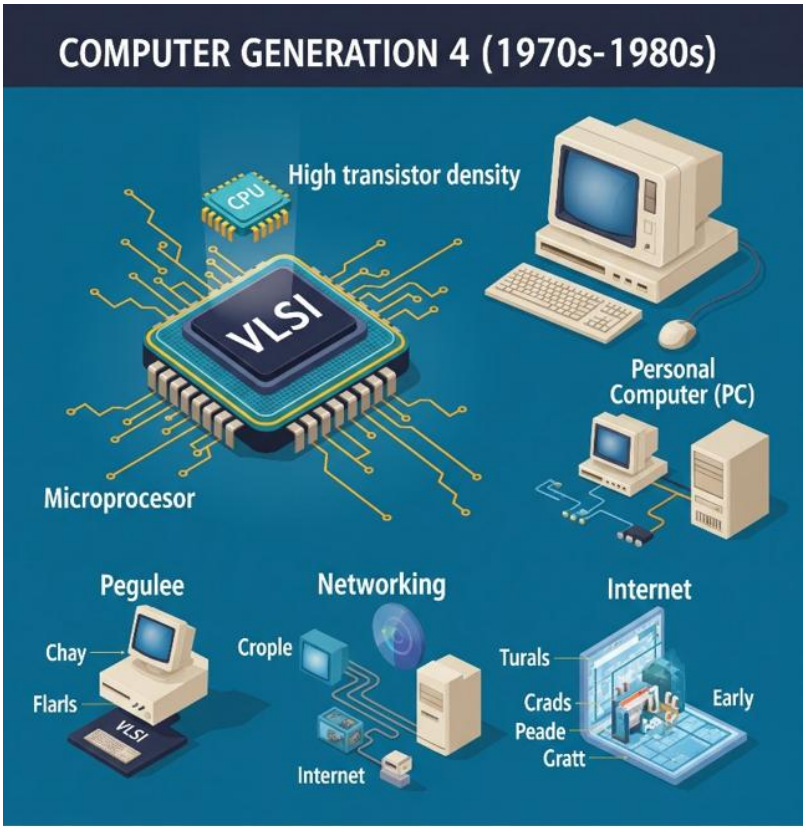
รูปที่ 1-7 การพัฒนาเทคโนโลยีและภาษาโปรแกรมในคอมพิวเตอร์ยุคที่ 2



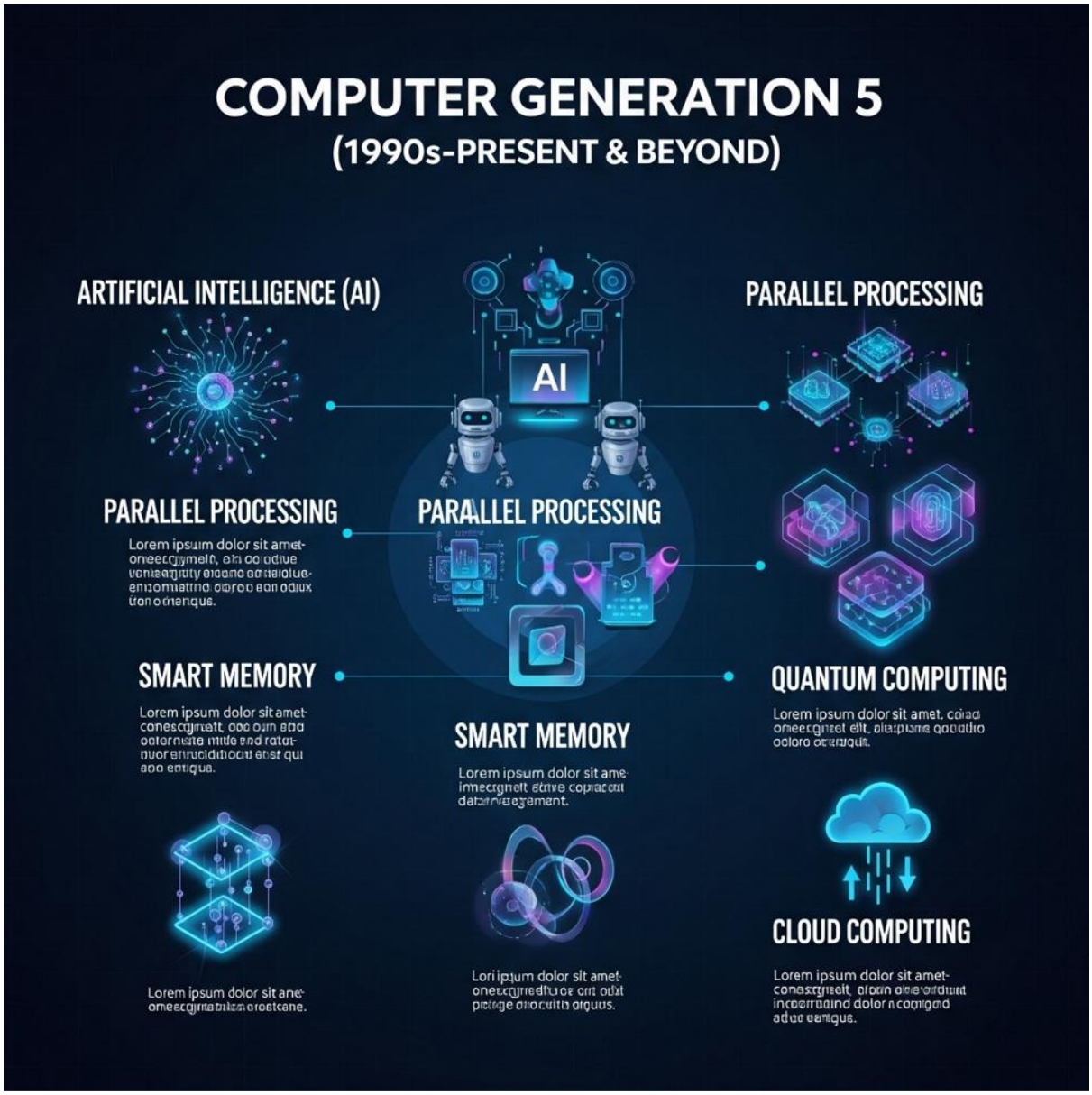
รูปที่ 1-8 การพัฒนาเทคโนโลยีคอมพิวเตอร์วงจรเบ็ดเสร็จ (Integrated Circuit)

ยุคที่ 3 (ทศวรรษ 1960-1970s): เทคโนโลยี **แผงวงจรรวม (Integrated Circuit - IC)** คือ ความก้าวหน้าสำคัญในยุคนี้ IC คือการนำทรานซิสเตอร์ ไดโอด ตัวต้านทาน และส่วนประกอบอิเล็กทรอนิกส์อื่นๆ จำนวนมากมารวมกันบนแผ่นซิลิคอนขนาดเล็ก ทำให้คอมพิวเตอร์มีขนาดเล็กลงไปอีกระดับ (เช่น คอมพิวเตอร์เมนเฟรมขนาดใหญ่เริ่มมีขนาดเล็กลง) มีประสิทธิภาพสูงขึ้น และราคาถูกลง ระบบปฏิบัติการ (Operating Systems) เริ่มมีความซับซ้อนมากขึ้น ทำให้สามารถจัดการทรัพยากรและรันโปรแกรมได้หลายโปรแกรมพร้อมกัน

ยุคที่ 4 (ทศวรรษ 1970-1980s): ยุคนี้โดดเด่นด้วยการพัฒนา **Very Large Scale Integration (VLSI)** ซึ่งสามารถบรรจุทรานซิสเตอร์หลายแสนถึงหลายล้านตัวลงใน IC เพียงชิ้นเดียว นำไปสู่การกำเนิดของ **ไมโครโปรเซสเซอร์ (Microprocessor)** ซึ่งเป็น CPU ขนาดเล็กที่รวมฟังก์ชันการประมวลผลทั้งหมดไว้ในชิปเดียว สิ่งนี้ทำให้เกิดคอมพิวเตอร์ส่วนบุคคล (Personal Computer - PC) ที่มีราคาจับต้องได้และเข้าถึงได้ง่ายขึ้น การเชื่อมต่อเครือข่ายคอมพิวเตอร์ (Networking) และอินเทอร์เน็ตเริ่มมีการพัฒนาและใช้งานในวงกว้าง



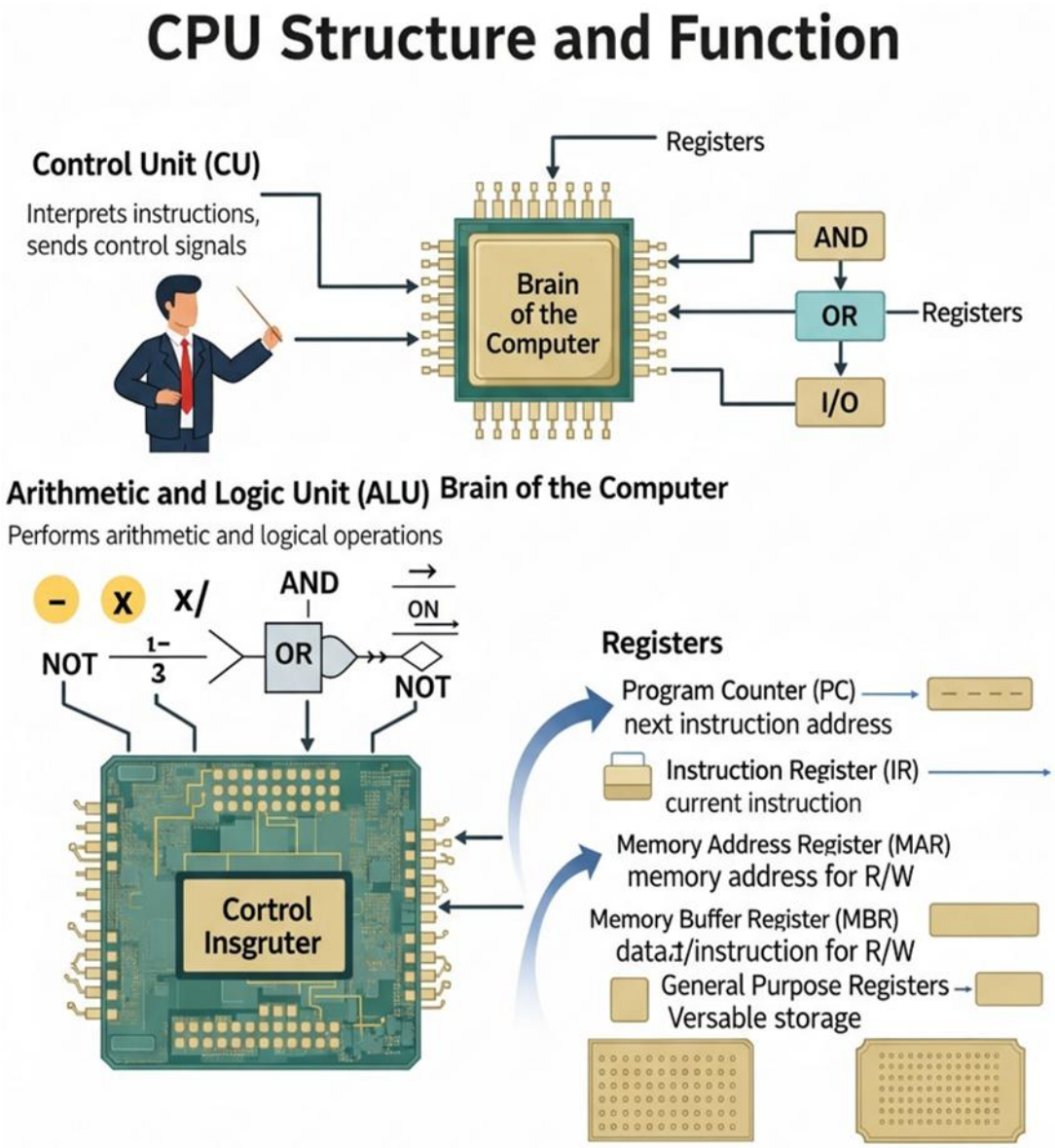
รูปที่ 1-9 การพัฒนาคอมพิวเตอร์ยุคไมโครโพรเซสเซอร์และเครือข่ายอินเทอร์เน็ต



รูปที่ 1-10 คอมพิวเตอร์ยุคปัญญาประดิษฐ์และเทคโนโลยีแห่งอนาคต

ยุคที่ 5 (ทศวรรษ 1980s - ปัจจุบัน): คอมพิวเตอร์ในยุคนี้เน้นการพัฒนาด้าน **ปัญญาประดิษฐ์ (Artificial Intelligence - AI)**, ระบบผู้เชี่ยวชาญ (Expert Systems), การประมวลผลแบบขนาน (Parallel Processing) ที่ใช้ CPU หลายตัวทำงานพร้อมกัน และการพัฒนาเครือข่ายอินเทอร์เน็ตที่ก้าวหน้าไปอย่างรวดเร็ว ทำให้เกิดการประมวลผลแบบคลาวด์ (Cloud Computing) และอุปกรณ์อัจฉริยะต่างๆ ที่เชื่อมโยงถึงกันอย่างมากมาย

3. โครงสร้างและหน้าที่ของหน่วยประมวลผลกลาง (CPU Structure and Function)

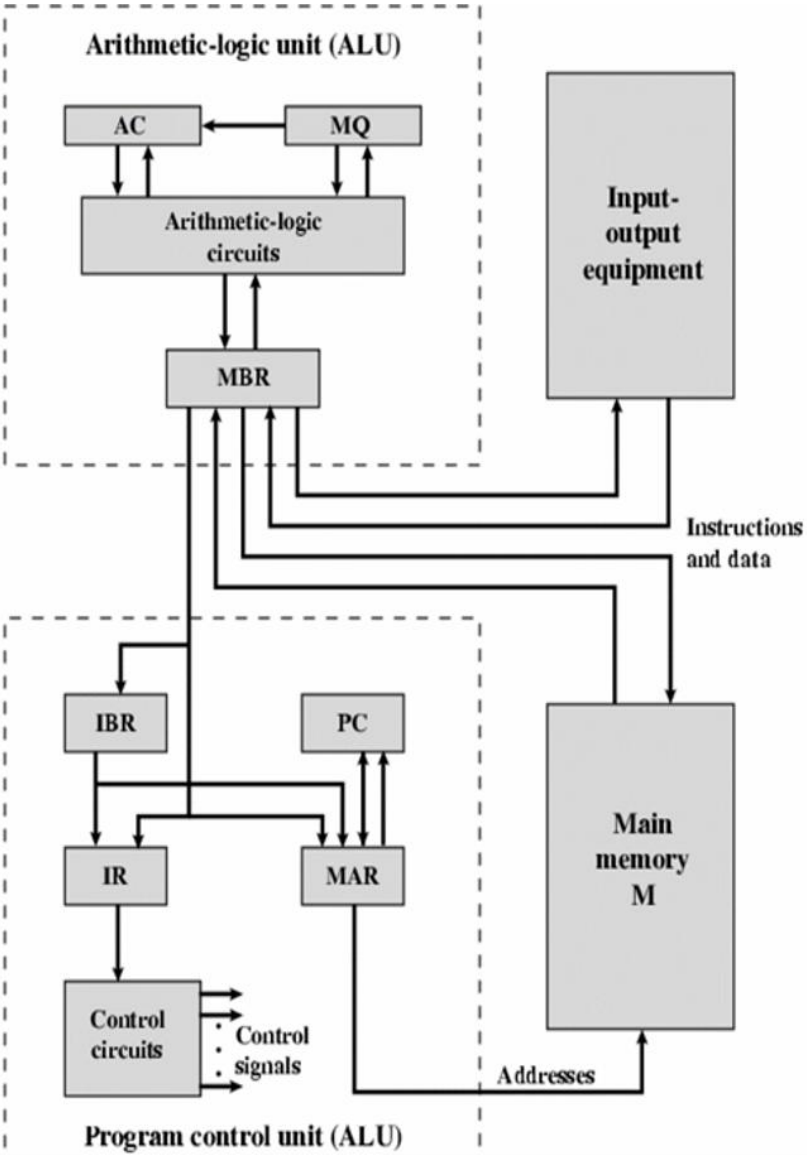


รูปที่ 1-11 โครงสร้างภายในและหน้าที่การทำงานของหน่วยประมวลผลกลาง (CPU)

หน่วยประมวลผลกลาง (CPU): คือ "สมอง" ของคอมพิวเตอร์ ทำหน้าที่ตีความและประมวลผลคำสั่งทั้งหมดของโปรแกรม รวมถึงการจัดการข้อมูล

หน่วยควบคุม (Control Unit - CU): เป็นส่วนที่สำคัญที่สุดใน CPU ทำหน้าที่เป็นผู้ประสานงานหลัก คอยตีความคำสั่งที่ถูกดึงมาจากหน่วยความจำ และส่งสัญญาณควบคุมไปยังส่วนประกอบอื่นๆ ของ CPU (เช่น ALU, Registers) และอุปกรณ์ I/O เพื่อให้ทำงานตามที่คำสั่งระบุ เหมือนผู้จัดการโครงการที่คอยสั่งการและจัดลำดับงาน

หน่วยคำนวณและตรรกะ (Arithmetic and Logic Unit - ALU): เป็นส่วนที่รับผิดชอบการดำเนินการทางคณิตศาสตร์ทั้งหมด (บวก ลบ คูณ หาร) และการดำเนินการทางตรรกะ (AND, OR, NOT, XOR, การเปรียบเทียบค่าต่างๆ) ผลลัพธ์ของการดำเนินการเหล่านี้จะถูกเก็บไว้ในรีจิสเตอร์ชั่วคราว



รูปที่ 1-12 แผนผังโครงสร้างสถาปัตยกรรมคอมพิวเตอร์แบบ IAS (Institute for Advanced Study)

รีจิสเตอร์ (Registers): เป็นหน่วยความจำขนาดเล็กพิเศษที่อยู่ภายใน CPU โดยตรง มีความเร็วในการเข้าถึงข้อมูลสูงที่สุดในระบบคอมพิวเตอร์ ใช้สำหรับเก็บข้อมูลชั่วคราวที่ CPU กำลังทำงานอยู่ เช่น

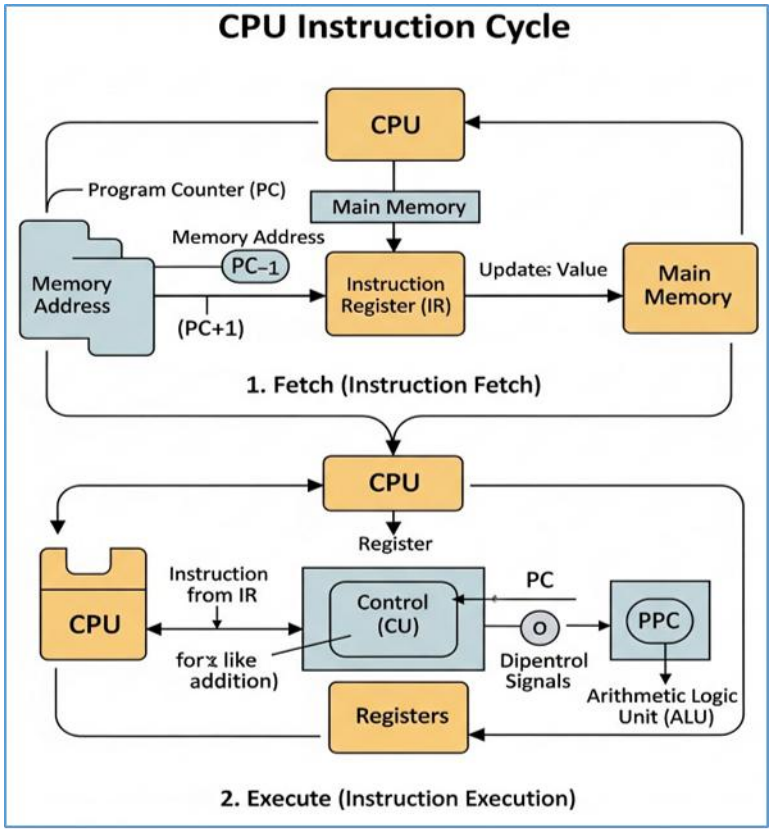
Program Counter (PC): เก็บที่อยู่ของคำสั่งถัดไปที่ CPU จะต้องดึงมาประมวลผล (Instruction Address Calculation)

Instruction Register (IR): เก็บคำสั่งปัจจุบันที่ CPU กำลังประมวลผล (Instruction Fetch)

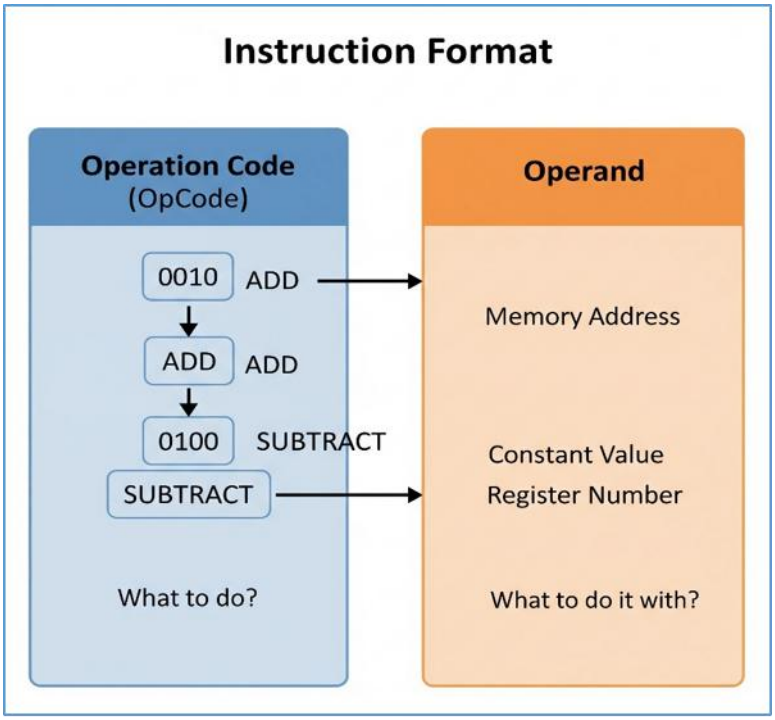
Memory Address Register (MAR): เก็บที่อยู่ของข้อมูลหรือคำสั่งในหน่วยความจำที่ CPU ต้องการอ่านหรือเขียน

Memory Buffer Register (MBR): เก็บข้อมูลหรือคำสั่งที่ถูกอ่านมาจากหรือกำลังจะเขียนไปยังหน่วยความจำ

General Purpose Registers: รีจิสเตอร์ทั่วไปที่ใช้สำหรับเก็บข้อมูลระหว่างการคำนวณ



รูปที่ 1-13 วงจรการทำงานของคำสั่งใน CPU (Fetch and Execute Cycle)



รูปที่ 1-14 รูปแบบและส่วนประกอบของคำสั่งคอมพิวเตอร์ (Instruction Format)

วงรอบคำสั่งของ CPU (Instruction Cycle): เป็นกระบวนการวนซ้ำที่ CPU ใช้ในการประมวลผลคำสั่งแต่ละคำสั่งในโปรแกรม โดยมี 2 ขั้นตอนหลักคือ:

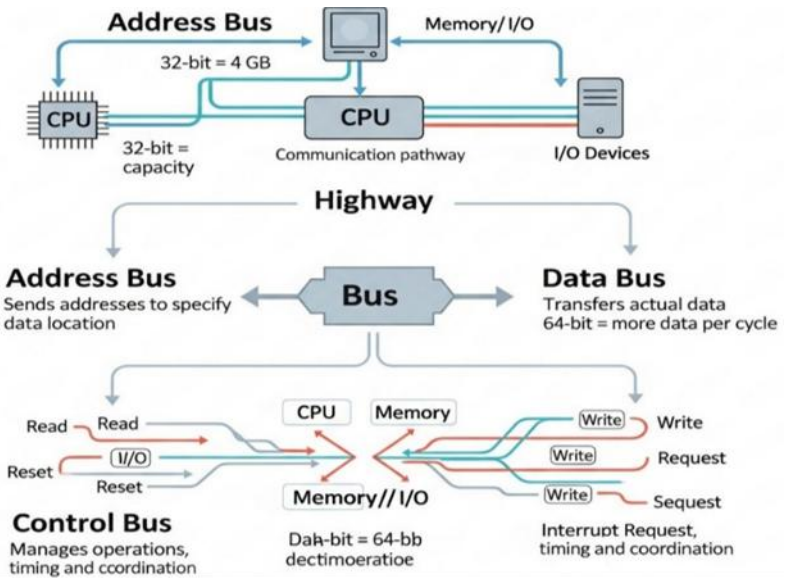
Fetch (การดึงคำสั่ง): CPU ดึงคำสั่งจากหน่วยความจำหลักมาเก็บไว้ใน Instruction Register (IR) โดยใช้ค่าที่อยู่ใน Program Counter (PC) เป็นตัวระบุตำแหน่ง หลังจากดึงคำสั่งแล้ว ค่าใน PC จะถูกอัปเดตเพื่อชี้ไปยังคำสั่งถัดไป

Execute (การปฏิบัติการคำสั่ง): Control Unit จะตีความคำสั่งที่อยู่ใน IR ว่าเป็นคำสั่งประเภทใด (เช่น บวก ลบ ย้ายข้อมูล) และส่งสัญญาณควบคุมไปยัง ALU หรือส่วนอื่นๆ ที่เกี่ยวข้องเพื่อดำเนินการตามคำสั่งนั้นๆ เช่น ถ้าเป็นคำสั่งบวก ALU ก็จะทำการบวกเลขตามที่ระบุในคำสั่ง

รูปแบบของ Instruction (Instruction Format): คำสั่งที่ CPU เข้าใจประกอบด้วย 2 ส่วนหลัก:

- **Operation Code (OpCode):** เป็นรหัสที่บอกว่า CPU ต้องทำ "อะไร" เช่น รหัส '0010' อาจหมายถึง "ADD", '0100' อาจหมายถึง "SUBTRACT" เป็นต้น
- **Operand:** เป็นส่วนที่บอกว่า "กระทำกับอะไร" เช่น อาจเป็นที่อยู่ของข้อมูลในหน่วยความจำ, ค่าคงที่, หรือหมายเลขของรีจิสเตอร์ที่จะใช้ในการประมวลผล

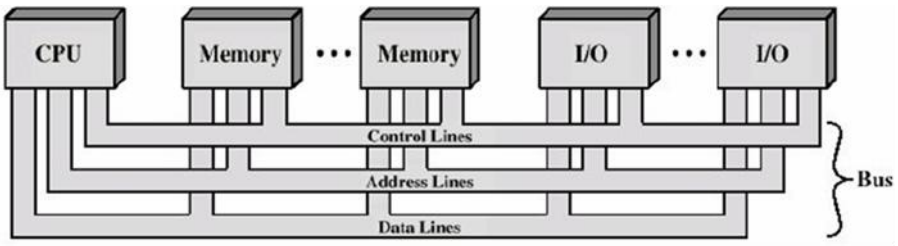
4. การเชื่อมต่อระบบบัสคอมพิวเตอร์ (System Interconnection - BUS)



รูปที่ 1-15 ระบบบัสและการเชื่อมต่อระหว่างองค์ประกอบภายใน (System Bus)

บัส (Bus): เป็นช่องทางสื่อสารที่ใช้ในการส่งข้อมูล แอดเดรส และสัญญาณควบคุม ระหว่างส่วนประกอบต่างๆ ของคอมพิวเตอร์ เปรียบเสมือนถนนที่เชื่อมโยงทุกส่วนเข้าด้วยกัน บัสแบ่งออกเป็น 3 ประเภทหลักตามหน้าที่:

Address Bus (บัสที่อยู่): เป็นบัสแบบทิศทางเดียว (Unidirectional) จาก CPU ไปยังหน่วยความจำหรืออุปกรณ์ I/O ใช้สำหรับส่งสัญญาณแอดเดรส เพื่อระบุตำแหน่งของข้อมูลที่ CPU ต้องการอ่านหรือเขียนในหน่วยความจำ หรือระบุพอร์ตของอุปกรณ์ I/O ที่ต้องการติดต่อ ยิ่ง Address Bus มีจำนวนบิตมากเท่าใด CPU ก็ยิ่งสามารถอ้างอิงตำแหน่งหน่วยความจำได้มากเท่านั้น (เช่น 32-bit Address Bus สามารถเข้าถึง 232 ไบต์ หรือ 4 GB)

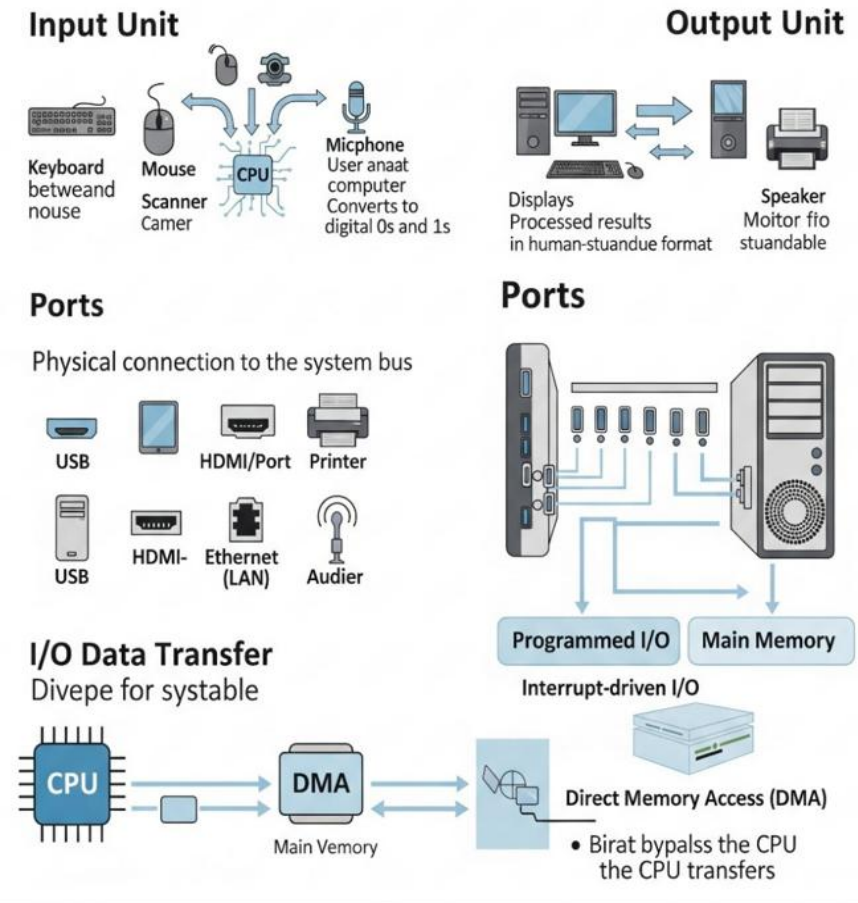


รูปที่ 1-16 โครงสร้างสายสัญญาณภายในบัสควบคุม บัสที่อยู่ และบัสข้อมูล

Data Bus (บัสข้อมูล): เป็นบัสแบบสองทิศทาง (Bidirectional) ใช้สำหรับรับส่งข้อมูลจริงระหว่าง CPU กับหน่วยความจำ หรือ CPU กับอุปกรณ์ I/O ในแต่ละรอบสัญญาณนาฬิกา ยิ่ง Data Bus มีจำนวนบิตมากเท่าใด (เช่น 32-bit, 64-bit) ก็จะสามารถส่งข้อมูลได้ในปริมาณที่มากขึ้นในคราวเดียว ทำให้การทำงานเร็วขึ้น

Control Bus (บัสควบคุม): เป็นบัสที่ใช้รับส่งสัญญาณควบคุมการทำงานต่างๆ เช่น สัญญาณ Read (บอกให้อ่านข้อมูล), Write (บอกให้เขียนข้อมูล), Reset (รีเซ็ตระบบ), Interrupt Request (สัญญาณขัดจังหวะจากอุปกรณ์ I/O) ซึ่งเป็นสัญญาณที่ช่วยควบคุมจังหวะและลำดับการทำงานของส่วนประกอบต่างๆ ให้เป็นไปอย่างถูกต้องและเป็นระเบียบ

5. การทำงานของหน่วยรับและส่งข้อมูล (Input/Output Unit)



รูปที่ 1-17 หน่วยรับ/แสดงผล พอร์ตเชื่อมต่อ และกลไกการรับส่งข้อมูล (I/O Unit)

หน่วยรับข้อมูลเข้า (Input Unit): ทำหน้าที่เป็นสะพานเชื่อมระหว่างผู้ใช้งานกับคอมพิวเตอร์ โดยรับข้อมูลจากภายนอก (เช่น การกดปุ่มบนคีย์บอร์ด, การเคลื่อนไหวของเมาส์, การสแกนภาพ) และแปลงข้อมูลเหล่านั้นให้อยู่ในรูปแบบดิจิทัล (0s และ 1s) ที่ CPU สามารถเข้าใจและประมวลผลได้ อุปกรณ์ Input Unit ที่คุ้นเคยได้แก่ Keyboard, Mouse, Microphone, Scanner, Webcam

หน่วยส่งข้อมูลออก (Output Unit): ทำหน้าที่นำผลลัพธ์ที่ CPU ได้ประมวลผลแล้ว ออกมาแสดงให้ผู้ใช้งานเห็นในรูปแบบที่มนุษย์เข้าใจ เช่น การแสดงผลบนหน้าจอ (Monitor), การพิมพ์ลงบนกระดาษ (Printer), การสร้างเสียง (Speaker)

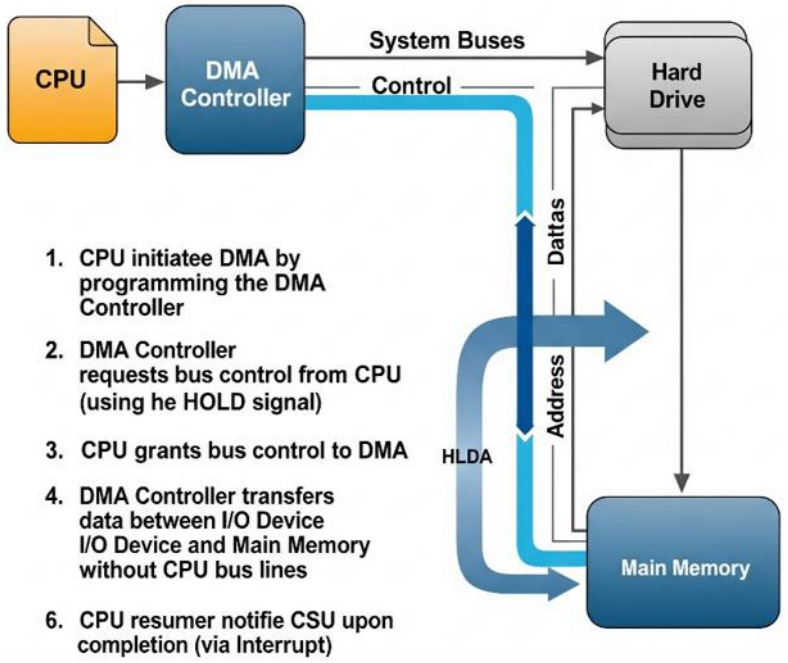
พอร์ต (Port): คือจุดเชื่อมต่อทางกายภาพบนคอมพิวเตอร์ ที่อนุญาตให้อุปกรณ์ I/O ภายนอกสามารถเสียบเชื่อมต่อกับระบบบัสภายในได้ พอร์ตมีหลายประเภทและมีหน้าที่เฉพาะเจาะจง เช่น

USB Port: พอร์ตอเนกประสงค์ที่ใช้สำหรับเชื่อมต่ออุปกรณ์หลากหลายประเภท เช่น เมาส์ คีย์บอร์ด Flash Drive เครื่องพิมพ์ กล้อง

HDMI/DisplayPort: สำหรับเชื่อมต่อจอภาพหรือโปรเจคเตอร์

Ethernet (LAN) Port: สำหรับเชื่อมต่อเครือข่ายอินเทอร์เน็ตแบบใช้สาย

Audio Jack: สำหรับเชื่อมต่อลำโพงหรือหูฟัง



รูปที่ 1-18 ขั้นตอนการทำงานของกระบวนการ Direct Memory Access (DMA)

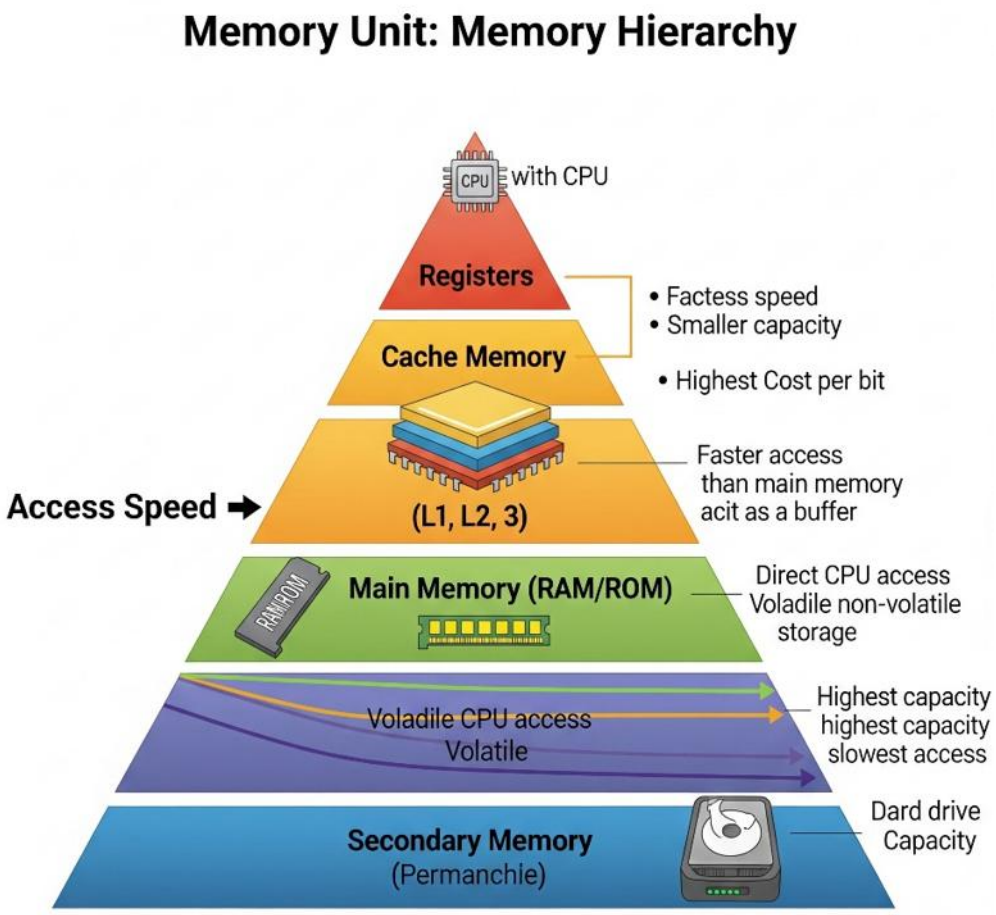
การโอนถ่ายข้อมูล I/O: ข้อมูลจาก I/O จะถูกโอนถ่ายไปยังหน่วยความจำหลัก และจากหน่วยความจำหลักไปยัง I/O โดยมีกลไกควบคุมการโอนถ่ายข้อมูลหลายวิธี เช่น Programmed I/O, Interrupt-driven I/O, และ Direct Memory Access (DMA) ซึ่ง DMA เป็นวิธีที่ช่วยลดภาระ CPU ในการโอนถ่ายข้อมูลปริมาณมากได้ดีที่สุด

ขั้นตอนการทำงาน (Process Flow)

1. CPU สั่งงาน: CPU ส่งคำสั่งไปที่ DMA Controller บอกตำแหน่งในหน่วยความจำ (Address) และจำนวนข้อมูล (Count)
2. DMA ขอใช้บัส: DMA จะส่งสัญญาณขอใช้ System Bus (Bus Request)
3. โอนถ่ายข้อมูล: เมื่อได้รับอนุญาต DMA จะดึงข้อมูลจาก I/O ยัดลง Memory โดยตรง (Direct) โดยไม่ผ่าน CPU เลย
4. CPU ทำงานขนาน: ในช่วงเวลานี้ CPU สามารถประมวลผลคำสั่งอื่นๆ ที่ไม่จำเป็นต้องใช้ System Bus ได้ (หรือถ้า CPU จำเป็นต้องใช้ Bus ช่วงนั้น อาจจะต้องรอ DMA นิดหน่อย เรียกว่า Cycle Stealing แต่โดยรวมก็ยังเร็วกว่าต้องมาขนเอง)
5. เสร็จสิ้น: เมื่อครบตามจำนวน DMA จะส่ง Interrupt บอก CPU ว่างานเสร็จแล้ว

สรุป: *DMA ดีที่สุดสำหรับการโอนถ่ายข้อมูลปริมาณมาก (เช่น อ่านไฟล์จาก Hard Disk, ข้อมูลภาพและเสียง) ช่วยลดภาระ CPU จากที่ต้องถูกขัดจังหวะทุกๆ ไบต์ (แบบ Interrupt-driven) เหลือเพียงถูกขัดจังหวะแค่ตอนเริ่มและตอนจบ*

6. การทำงานของหน่วยความจำ (Memory Unit)



รูปที่ 1-19 ลำดับชั้นของหน่วยความจำ (Memory Hierarchy Pyramid)

หน่วยความจำ (Memory Unit): เป็นส่วนสำคัญที่ทำหน้าที่เก็บชุดคำสั่งของโปรแกรมและข้อมูลต่างๆ ที่ CPU กำลังทำงานอยู่ หรือที่เตรียมจะส่งไปให้ CPU ประมวลผล รวมถึงเก็บผลลัพธ์ชั่วคราวจากการประมวลผล ก่อนที่จะส่งไปยังหน่วยแสดงผล หรือบันทึกลงหน่วยความจำสำรอง

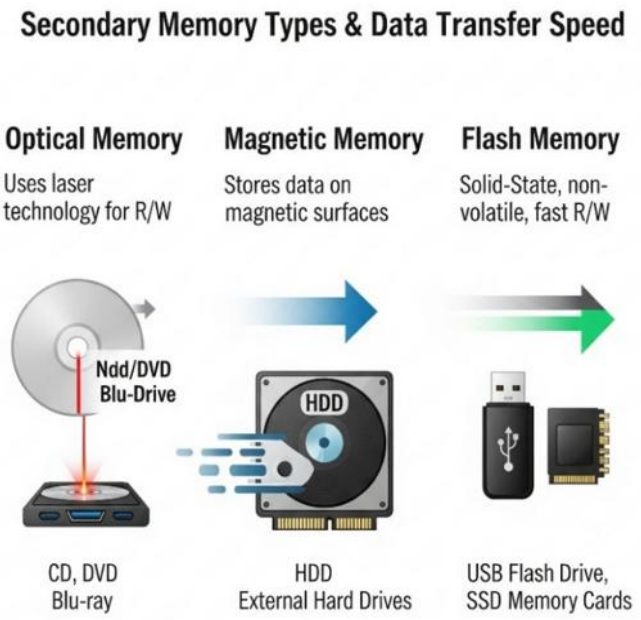
ลำดับชั้นหน่วยความจำ (Memory Hierarchy): เนื่องจากข้อจำกัดด้านความเร็ว, ความจุ, และต้นทุนหน่วยความจำในระบบคอมพิวเตอร์จึงถูกจัดเรียงเป็นลำดับชั้น โดยมีเป้าหมายเพื่อให้ CPU สามารถเข้าถึงข้อมูลที่ต้องการได้อย่างรวดเร็วที่สุดเท่าที่จะเป็นไปได้ ในขณะที่ยังคงรักษาสมดุลด้านความจุและต้นทุน ดังนี้:

รีจิสเตอร์ (Registers): อยู่ภายใน CPU เอง เป็นหน่วยความจำที่เข้าถึงเร็วที่สุด แพงที่สุด และมีความจุน้อยที่สุด ใช้เก็บข้อมูลที่ CPU กำลังทำงานอยู่ทันที

หน่วยความจำแคช (Cache Memory): เป็นหน่วยความจำขนาดเล็กที่เร็วกว่าหน่วยความจำหลัก แต่ช้ากว่ารีจิสเตอร์ ทำหน้าที่เป็นบัฟเฟอร์เก็บสำเนาข้อมูลและคำสั่งที่ CPU มีแนวโน้มจะใช้บ่อยๆ (จากหน่วยความจำหลัก) เพื่อลดเวลาในการเข้าถึงข้อมูลจากหน่วยความจำหลักที่มีความเร็วช้ากว่ามาก แคชแบ่งออกเป็นหลายระดับ เช่น L1, L2, L3 โดย L1 จะเร็วที่สุดและอยู่ใกล้ CPU ที่สุด

หน่วยความจำหลัก (Main Memory): เรียกว่า RAM (Random Access Memory) เป็นหน่วยความจำที่ CPU สามารถเข้าถึงข้อมูลได้โดยตรง เป็นที่เก็บโปรแกรมและข้อมูลที่กำลังทำงานอยู่ ข้อมูลใน RAM จะหายไปเมื่อปิดเครื่อง (volatile) นอกจากนี้ยังมี ROM (Read-Only Memory) ที่เก็บเฟิร์มแวร์หรือโปรแกรมเริ่มต้นเครื่อง ซึ่งไม่สามารถเขียนทับได้ง่ายๆ และข้อมูลไม่หายไปเมื่อปิดเครื่อง (non-volatile)

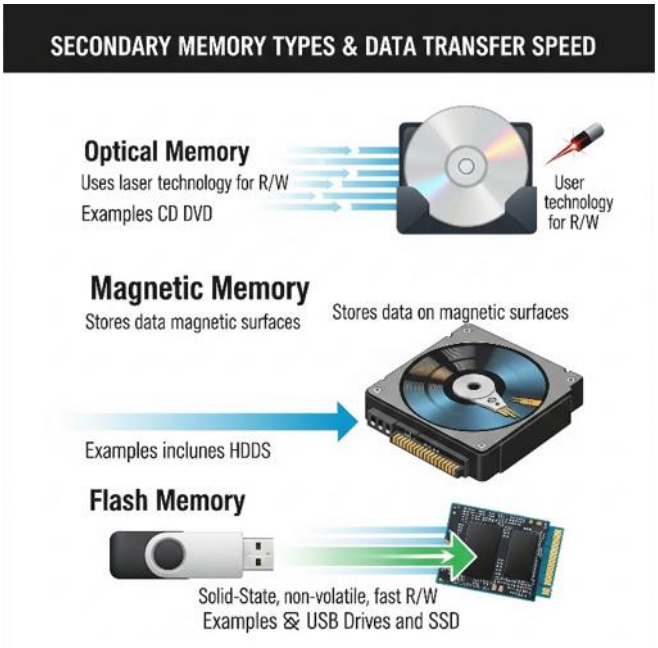
หน่วยความจำสำรอง (Secondary Memory): เป็นหน่วยความจำที่ใช้สำหรับจัดเก็บข้อมูลและโปรแกรมในระยะยาวอย่างถาวร ไม่ต้องใช้พลังงานไฟฟ้าในการเก็บรักษาข้อมูล (non-volatile) มีความจุสูงที่สุดแต่มีความเร็วในการเข้าถึงช้าที่สุดเมื่อเทียบกับหน่วยความจำชั้นบน



รูปที่ 1-21 การเปรียบเทียบประสิทธิภาพและความเร็วในการรับส่งข้อมูลของหน่วยความจำสำรอง

ประเภทของหน่วยความจำสำรอง (Secondary Memory Types):

หน่วยความจำแสง (Optical Memory): ใช้เทคโนโลยีเลเซอร์ในการอ่านและเขียนข้อมูล เช่น CD (Compact Disc), DVD (Digital Versatile Disc), และ Blu-ray Disc ซึ่งแต่ละชนิดมีความจุและเทคโนโลยีเลเซอร์ที่แตกต่างกัน



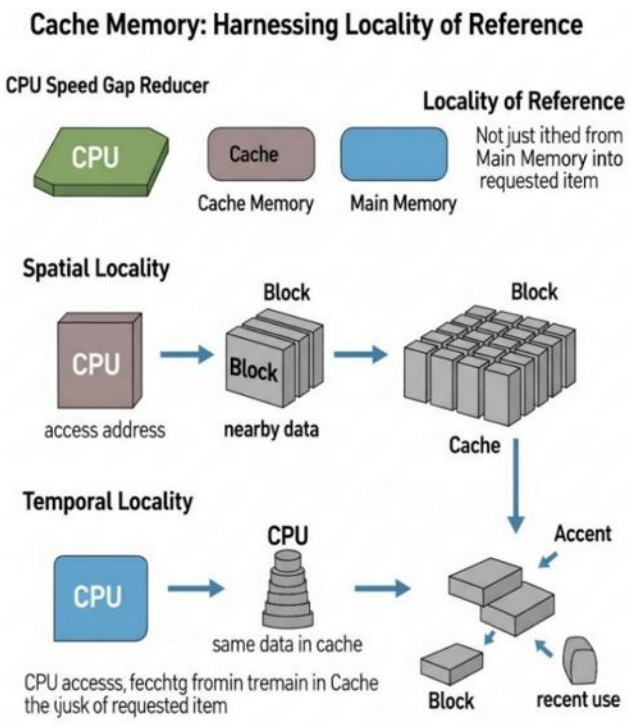
รูปที่ 1-20 ความเร็วในการส่งข้อมูลของหน่วยความจำสำรอง

หน่วยความจำแม่เหล็ก (Magnetic Memory): ใช้หลักการบันทึกข้อมูลลงบนสารแม่เหล็ก เช่น Hard Disk Drive (HDD) ซึ่งยังคงเป็นสื่อเก็บข้อมูลหลักในคอมพิวเตอร์ทั่วไป และ Floppy Disk (ปัจจุบันไม่ค่อยได้ใช้แล้ว)

หน่วยความจำแฟลช (Flash Memory): เป็นหน่วยความจำแบบ Solid-State (ไม่มีชิ้นส่วนเคลื่อนไหว) ที่สามารถลบและเขียนซ้ำได้หลายครั้ง เก็บข้อมูลได้แม้ไม่มีไฟฟ้าเลี้ยง (non-volatile) มีความเร็วในการเข้าถึงข้อมูลเร็วกว่า Magnetic Memory มาก นิยมใช้ใน Flash Drive (USB drive), Solid State Drive (SSD) ซึ่งกำลังเข้ามาแทนที่ HDD, และ Memory Card ในกล้องดิจิทัล/โทรศัพท์มือถือ

7. หน่วยความจำแคช (Cache Memory)

หลักการทำงานของ Cache Memory: แคชเป็นส่วนสำคัญที่ช่วยลด "ช่องว่างความเร็ว" (Speed Gap) ระหว่าง CPU ที่เร็วมากกับหน่วยความจำหลัก (RAM) ที่ช้ากว่า การทำงานของแคชอาศัยหลักการ **Locality of Reference (หลักการอ้างอิงเฉพาะที่)** ซึ่งมี 2 ประเภท:

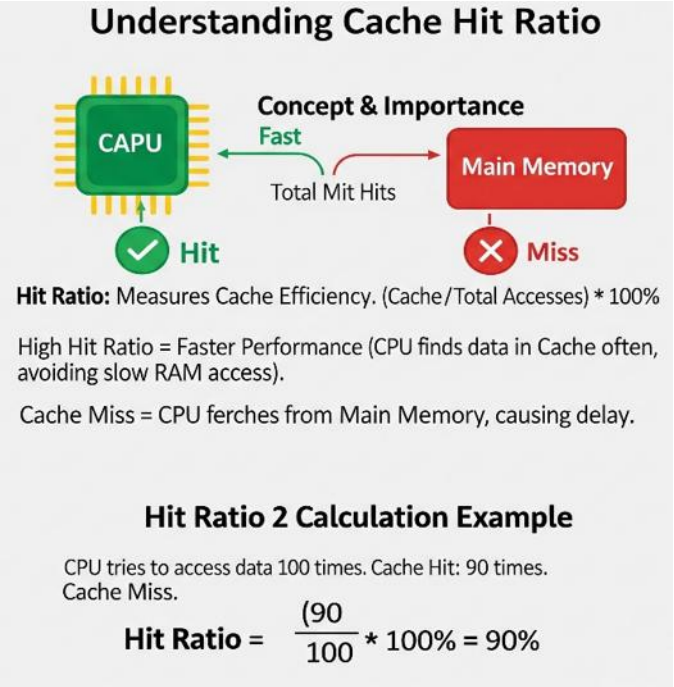


รูปที่ 1-22 การทำงานของหน่วยความจำแคช และหลักการ Locality of Reference

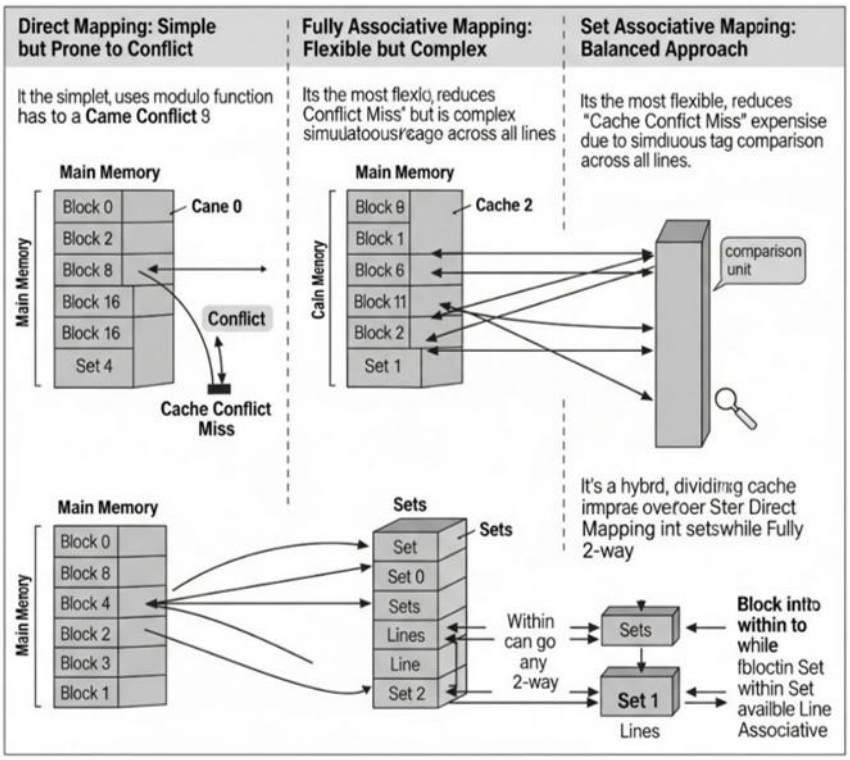
Spatial Locality (การอ้างอิงเฉพาะที่เชิงพื้นที่): หาก CPU เข้าถึงข้อมูลที่ตำแหน่งใดตำแหน่งหนึ่ง มีแนวโน้มสูงที่จะเข้าถึงข้อมูลที่อยู่ "ใกล้เคียง" กับตำแหน่งนั้นในอนาคตอันใกล้ (เช่น เมื่ออ่านข้อมูลในอาร์เรย์ ลูปที่เข้าถึงสมาชิกถัดไป) แคชจึงมักจะดึงข้อมูลมาเป็น "บล็อก" ขนาดใหญ่ (Block) เพื่อให้ครอบคลุมข้อมูลที่อยู่ติดกัน

Temporal Locality (การอ้างอิงเฉพาะที่เชิงเวลา): หาก CPU เข้าถึงข้อมูลที่ตำแหน่งใดตำแหน่งหนึ่ง มีแนวโน้มสูงที่จะกลับมาเข้าถึงข้อมูล "เดิม" นั้นซ้ำอีกครั้งในอนาคตอันใกล้ (เช่น ตัวแปรที่ใช้บ่อยๆ ในลูป) แคชจึงเก็บข้อมูลที่ถูกเรียกใช้บ่อยๆ ไว้

Hit Ratio (อัตราส่วนการเข้าถึงแคชสำเร็จ): เป็นตัวชี้วัดประสิทธิภาพของแคช Hit Ratio = (จำนวนครั้งที่เข้าถึงแคชสำเร็จ / จำนวนครั้งที่เข้าถึงทั้งหมด) * 100% ถ้า Hit Ratio สูง หมายความว่า CPU หาข้อมูลเจอในแคชบ่อย ทำให้การทำงานโดยรวมเร็วขึ้น (เพราะไม่ต้องไปดึงจาก RAM ที่ช้ากว่า) ถ้าเกิด Cache Miss (หาข้อมูลไม่เจอในแคช) CPU จะต้องไปดึงข้อมูลจากหน่วยความจำหลัก ซึ่งทำให้เสียเวลา



รูปที่ 1-23 สูตรการคำนวณและตัวอย่างการหาค่า Cache Hit Ratio



รูปที่ 1-24 เทคนิคการจับคู่หน่วยความจำแบบ Direct, Associative และ Set Associative

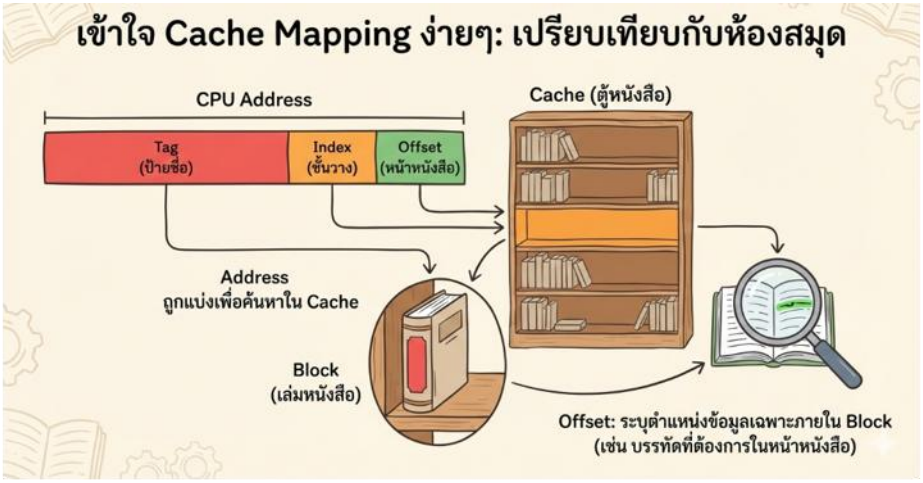
วิธีการเชื่อมโยงข้อมูลของหน่วยความจำแคช (Cache Mapping): เป็นวิธีการกำหนดว่าบล็อกข้อมูลจากหน่วยความจำหลักจะถูกนำไปเก็บไว้ที่ตำแหน่งใดในแคช มี 3 วิธีหลัก:

Direct Mapping: เป็นวิธีที่ง่ายที่สุด แต่มีโอกาสเกิด "Cache Conflict Miss" สูง กำหนดให้บล็อกหน่วยความจำหลักแต่ละบล็อกสามารถถูกแมปไปเก็บใน Line ของแคชได้เพียงตำแหน่งเดียวเท่านั้น หากหลายบล็อกจากหน่วยความจำหลักถูกแมปไปยัง Line เดียวกันในแคช ก็จะเกิดการชนกัน ทำให้บล็อกหนึ่งต้องถูกนำออกไปเมื่ออีกบล็อกหนึ่งถูกนำเข้ามา แม้ว่าแคชจะยังไม่เต็ม

Fully Associative Mappingเป็นวิธีที่ยืดหยุ่นที่สุด แต่ซับซ้อนและมีราคาแพง บล็อกหน่วยความจำหลักใดๆ สามารถถูกนำไปเก็บไว้ที่ตำแหน่งใดก็ได้ในแคช ทำให้ลดโอกาสเกิด Cache Conflict Miss ได้มากที่สุด แต่การค้นหาข้อมูลจะซับซ้อนเนื่องจากต้องเปรียบเทียบ Tag ของบล็อกที่ต้องการกับ Tag ของทุกบล็อกในแคชพร้อมกัน

Set Associative Mapping: เป็นการผสมผสานระหว่าง Direct Mapping และ Fully Associative Mapping โดยแบ่งแคชออกเป็น "ชุด" (Set) และแต่ละชุดประกอบด้วยบล็อกหลายบล็อก (เช่น 2-way, 4-way, 8-way) บล็อกหน่วยความจำหลักจะถูกแมปไปที่ Set ใด Set หนึ่งตายตัว (คล้าย Direct Mapping) แต่ภายใน Set นั้น บล็อกสามารถถูกวางในตำแหน่งใดก็ได้ (คล้าย Fully Associative Mapping) วิธีนี้ช่วยลดปัญหา Conflict Miss ได้ดีกว่า Direct Mapping และยังคงความซับซ้อนที่จัดการได้ดีกว่า Fully Associative Mapping

การคำนวณ Tag, Index/Set, และ Offset: ในการใช้งาน Cache Mapping CPU จะแบ่งแอดเดรสของข้อมูลออกเป็น 3 ส่วนเพื่อใช้ในการระบุตำแหน่ง เพราะฉะนั้นเมื่อ CPU ต้องการเข้าถึงข้อมูลในหน่วยความจำหลัก แอดเดรสของข้อมูลนั้นจะถูกแบ่งออกเป็น 3 ส่วนหลัก ๆ เพื่อใช้ในการจัดการและค้นหาข้อมูลในหน่วยความจำแคช



รูปที่ 1-25 การเปรียบเทียบส่วนประกอบของ Cache Address กับระบบห้องสมุด

1. Offset (Block Offset)

หน้าที่: ส่วนนี้จะระบุตำแหน่งของข้อมูล (ไบต์หรือเวิร์ด) ที่อยู่ภายในบล็อกของแคช

การกำหนดบิต: จำนวนบิตของ Offset ขึ้นอยู่กับขนาดของบล็อก (Block Size) ยิ่งบล็อกใหญ่ ก็ยิ่งต้องใช้จำนวนบิตในส่วน Offsetมากขึ้น เพื่อระบุตำแหน่งภายในบล็อกได้อย่างละเอียด ตัวอย่างเช่น ถ้า Block Size คือ 64 ไบต์ จะใช้ 6 บิต (2^6 = 64) สำหรับ Offset.

2. Index (หรือ Set Index) หน้าที่: ส่วนนี้จะระบุ "ตำแหน่ง" ของบรรทัด (Line) หรือชุด (Set) ภายในหน่วยความจำแคช ที่ข้อมูลจากหน่วยความจำหลักสามารถถูกจัดเก็บได้

การกำหนดบิต: จำนวนบิตของ Index ขึ้นอยู่กับจำนวน Line หรือ Set ทั้งหมดในแคช.

3.Tag หน้าที่: ส่วนนี้คือส่วนที่เหลือของแอดเดรสที่ไม่ได้ใช้เป็น Index หรือ Offset. Tag ทำหน้าที่เป็น "ป้ายกำกับ" ที่ใช้ยืนยันว่าบล็อกข้อมูลที่พบในแคชนั้น เป็นบล็อกที่ถูกต้องตรงกับแอดเดรสที่ CPU ต้องการหรือไม่.

การตรวจสอบ: เมื่อ CPU ต้องการข้อมูล จะใช้ Index เพื่อไปยังตำแหน่งที่คาดว่าข้อมูลจะอยู่ จากนั้นจะนำค่า Tag ที่อยู่ในแอดเดรสมาเปรียบเทียบกับค่า Tag ที่จัดเก็บอยู่พร้อมกับข้อมูลในแคช Line นั้น หากตรงกัน แสดงว่าเกิด Cache Hit และข้อมูลที่ต้องการอยู่ที่นั่น. หากไม่ตรงกันหรือไม่พบบล็อก (Valid Bit เป็น 0) แสดงว่าเกิด Cache Miss และต้องไปดึงข้อมูลจากหน่วยความจำหลักมา.



รูปที่ 1-26 โครงสร้างบิตข้อมูลใน Memory Address สำหรับการระบุตำแหน่งในแคช

แนวคิด (Conceptual Diagram) จากรูป สามารถอธิบายการการแบ่งแอดเดรสได้ดังนี้:

ตัวอย่างการทำงานแบบง่ายๆ:

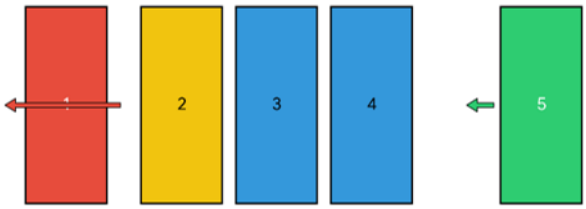
สมมติว่า CPU ต้องการข้อมูลที่แอดเดรสหนึ่ง

1. CPU จะใช้ **Offset** เพื่อระบุว่าข้อมูลที่ต้องการอยู่ในไบต์ที่เท่าไหร่ของบล็อกข้อมูลนั้น
2. CPU จะใช้ **Index** เพื่อระบุว่าบล็อกข้อมูลนี้ควรจะไปหาที่ Line (หรือ Set) ใดในแคช
3. เมื่อไปถึง Line (หรือ Set) ที่ระบุโดย Index แล้ว CPU จะนำค่า **Tag** ที่ได้จากแอดเดรสมาเปรียบเทียบกับ Tag ที่เก็บอยู่ในแคช Line เพื่อยืนยันว่าเป็นข้อมูลที่ถูกต้องของบล็อกนั้นจริง

อัลกอริทึมการเปลี่ยนบล็อก (Replacement Algorithms):

การคำนวณอัลกอริทึมการเปลี่ยนบล็อก (Replacement Algorithms) ในหน่วยความจำแคชระบบคอมพิวเตอร์ของเรา หน่วยความจำแคช (Cache Memory) มีขนาดจำกัด เมื่อแคชเต็มและมีข้อมูลบล็อกใหม่จากหน่วยความจำหลักต้องการเข้ามาในแคช ระบบจะต้องตัดสินใจว่าจะนำบล็อกข้อมูลเก่าบล็อกไหนออกไปเพื่อเปิดพื้นที่ให้บล็อกใหม่เข้ามาได้ กระบวนการนี้เรียกว่า **Replacement Algorithm** ซึ่งมีหลายวิธีด้วยกัน แต่ละวิธีก็มีข้อดีข้อเสียต่างกันไป และเหมาะกับการใช้งานที่แตกต่างกัน

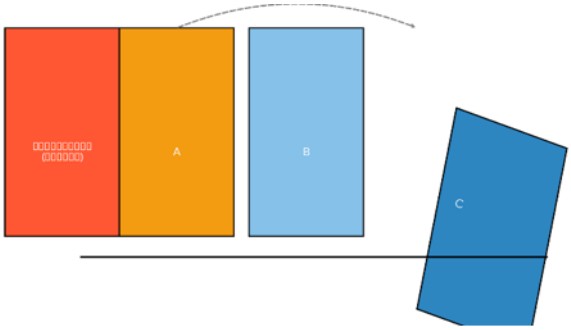
แบบที่ 1. FIFO (First-In First-Out): "ใครมาก่อน ไปก่อน!"



รูปที่ 1-27 กลไกการทดแทนข้อมูลในแคชแบบ First-In First-Out (FIFO)

- **แนวคิด:** อัลกอริทึมนี้จะเลือกบล็อกข้อมูลที่เข้ามาในแคช "นานที่สุด" ออกไปก่อนเป็นอันดับแรก ไม่ว่าบล็อกนั้นจะถูกใช้งานบ่อยแค่ไหน หรือเพิ่งถูกใช้งานไปเมื่อไหร่ก็ตาม
- **เปรียบเทียบง่ายๆ:** เหมือนกับการเข้าคิวซื้อของ ใครเข้าคิวก่อน ก็ได้ออกไปก่อน (ไม่ว่าจะซื้อของเสร็จนานแล้วหรือยังซื้อไม่เสร็จก็ตาม)
- **ข้อดี: เข้าใจง่ายและใช้งานง่าย:** ไม่ต้องมีกลไกซับซ้อนในการติดตามการใช้งาน
- **ข้อเสีย: ไม่ฉลาด:** อาจจะนำบล็อกที่ยังถูกใช้งานอยู่บ่อยๆ ออกไป ทำให้เกิด Cache Miss (พลาดแคช) บ่อยขึ้น
- **เหมาะสำหรับ:** ระบบที่ไม่ต้องการความซับซ้อนมากนัก

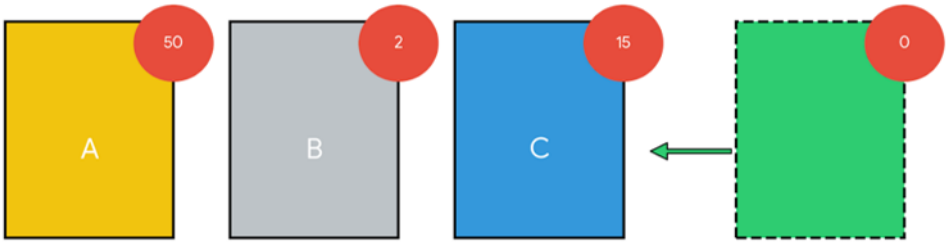
แบบที่ 2. LRU (Least Recently Used): "ใครไม่ได้ใช้มานานที่สุด ไปก่อน!"



รูปที่ 1-28 กลไกการทดแทนข้อมูลในแคชแบบ Least Recently Used (LRU)

- **แนวคิด:** อัลกอริทึมนี้จะเลือกบล็อกข้อมูลที่ "ไม่ได้ถูกใช้งานมานานที่สุด" (นับจากเวลาที่ถูกเข้าถึงครั้งสุดท้าย) ออกไปเพื่อเปิดพื้นที่ให้บล็อกใหม่
- **เปรียบเทียบง่ายๆ:** เหมือนกับของในตู้เย็น ถ้าไม่ได้หยิบมาใช้นานแล้ว ก็เป็นอันดับแรกที่จะถูกนำไปทิ้ง
- **ข้อดี: มีประสิทธิภาพสูง:** มักจะให้ผลลัพธ์ที่ดีกว่า FIFO เพราะสอดคล้องกับหลักการ **Temporal Locality** (ข้อมูลที่เพิ่งถูกใช้งานไป มีแนวโน้มที่จะถูกใช้งานอีกครั้งในอนาคตอันใกล้)
- **ข้อเสีย: ซับซ้อนในการจัดการ:** ต้องมีกลไกที่คอยติดตามว่าบล็อกแต่ละบล็อกถูกเข้าถึงครั้งสุดท้ายเมื่อไหร่ ซึ่งเพิ่มภาระการทำงาน
- **เหมาะสำหรับ:** ระบบที่ต้องการประสิทธิภาพสูง และยอมรับความซับซ้อนในการจัดการได้

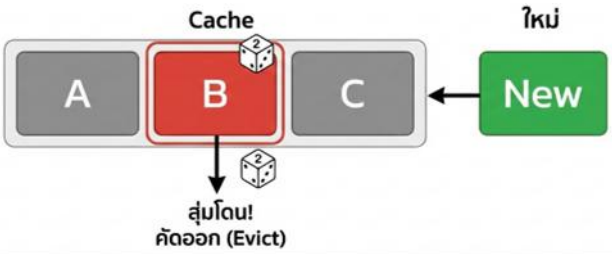
แบบที่ 3. LFU (Least Frequently Used): "ใครใช้น้อยที่สุด ไปก่อน!"



รูปที่ 1-29 กลไกการทดแทนข้อมูลแบบ Least Frequently Used (LFU)

- **แนวคิด:** อัลกอริทึมนี้จะเลือกบล็อกข้อมูลที่ "ถูกใช้งานน้อยที่สุด" (นับจากจำนวนครั้งที่ถูกเข้าถึง) ออกไป
- **เปรียบเทียบง่ายๆ:** เหมือนกับการเลือกของเล่นในกล่อง ของเล่นชิ้นไหนที่เด็กไม่ค่อยเล่น ก็จะถูกเก็บออกไป
- **ข้อดี: เน้นบล็อกที่ใช้งานบ่อย:** พยายามคงบล็อกที่ใช้งานบ่อยๆ ไว้ในแคช
- **ข้อเสีย: ซับซ้อนในการจัดการ:** ต้องมีตัวนับ (counter) สำหรับทุกบล็อกในแคช เพื่อติดตามจำนวนครั้งที่ถูกใช้งาน ซึ่งเป็นภาระที่สูง
- **ปัญหา "เริ่มต้น":** บล็อกที่เพิ่งเข้ามาใหม่ อาจจะยังไม่มีโอกาสถูกใช้งานบ่อยๆ ก็อาจจะถูกนำออกไปได้ง่ายๆ ทั้งที่อาจจะเป็นบล็อกสำคัญ
- **เหมาะสำหรับ:** การใช้งานเฉพาะทางที่ต้องการเน้นความถี่ในการใช้งานเป็นหลัก

แบบที่ 4. Random: "สุ่มไปเลย!"



รูปที่ 1-30 กลไกการทดแทนข้อมูลแบบสุ่ม (Random Replacement Policy)

- **แนวคิด:** อัลกอริทึมนี้จะเลือกบล็อกข้อมูลที่จะนำออกไปจากแคชแบบ "สุ่ม" โดยไม่มีหลักเกณฑ์ใดๆ ในการพิจารณาการใช้งานหรือเวลา
- **เปรียบเทียบง่ายๆ:** เหมือนกับการจับฉลาก ใครได้จับออกไป ก็คือออกไป
- **ข้อดี: ง่ายและรวดเร็วที่สุด:** ไม่ต้องมีกลไกการจัดการที่ซับซ้อน ไม่ต้องติดตามข้อมูลใดๆ
- **ข้อเสีย: ประสิทธิภาพไม่แน่นอน:** อาจจะนำบล็อกที่จำเป็นมากออกไป หรือบล็อกที่ไม่จำเป็นไว้ในแคช ทำให้ประสิทธิภาพต่ำกว่าวิธีอื่น
- **เหมาะสำหรับ:** ระบบที่ความง่ายในการใช้งานสำคัญกว่าประสิทธิภาพสูงสุด หรือในกรณีที่การใช้งานไม่เป็นรูปแบบที่ชัดเจน

นโยบายการเขียนข้อมูล (Write Policy) เมื่อมีการแก้ไขข้อมูลในแคช จะต้องมีนโยบายในการจัดการให้ข้อมูลในแคชและหน่วยความจำหลักมีความสอดคล้องกัน เมื่อ CPU มีการแก้ไขข้อมูลที่อยู่ในหน่วยความจำแคช (Cache Memory) สิ่งสำคัญคือข้อมูลนั้นจะต้องมีความสอดคล้องกับข้อมูลในหน่วยความจำหลัก (Main Memory) เสมอ เพื่อป้องกันความคลาดเคลื่อนของข้อมูลและให้แน่ใจว่าระบบทำงานได้อย่างถูกต้อง การจัดการความสอดคล้องนี้เรียกว่า **"นโยบายการเขียนข้อมูล" (Write Policy)** ซึ่งมี 2 วิธีหลักๆ ดังนี้

1. Write Through (เขียนผ่าน)

- **แนวคิด:** เมื่อใดก็ตามที่ CPU ทำการแก้ไขข้อมูลในแคช ข้อมูลที่ถูกแก้ไขนั้นจะ **ถูกเขียนกลับไปยังหน่วยความจำหลักด้วยทันที** ในเวลาเดียวกัน
- **การทำงานโดยละเอียด:**
  1. CPU เขียนข้อมูลใหม่ลงในบล็อกของแคช
  2. **พร้อมกันนั้น** หรือแทบจะในทันที ข้อมูลใหม่เดียวกันนี้จะถูกส่งไปเขียนทับข้อมูลในตำแหน่งที่สอดคล้องกันในหน่วยความจำหลักด้วย
- **ข้อดี: ข้อมูลสอดคล้องกันเสมอ:** ทำให้ข้อมูลในแคชและหน่วยความจำหลักเป็นค่าเดียวกันตลอดเวลา ลดความเสี่ยงที่ข้อมูลจะคลาดเคลื่อนหรือสูญหายหากเกิดระบบขัดข้อง
  กู้คืนข้อมูลได้ง่าย: หากแคชเกิดล้มเหลว ข้อมูลในหน่วยความจำหลักจะยังคงถูกต้อง
- **ข้อเสีย: ความเร็วในการเขียนช้าลง:** เนื่องจากทุกครั้งที่มีการเขียนข้อมูลลงแคช จะต้องมีการเขียนลงหน่วยความจำหลัก (RAM) ด้วย ทำให้ต้องเสียเวลารอการเข้าถึง RAM ซึ่งช้ากว่าแคชมาก และเป็นการใช้บัสข้อมูล (Data Bus) บ่อยครั้ง

สิ้นเปลืองแบนด์วิดท์ของบัส: การส่งข้อมูลไปยังหน่วยความจำหลักบ่อยครั้งทำให้บัสข้อมูลถูกใช้งานมากขึ้น ลดทอนประสิทธิภาพโดยรวมของระบบ



รูปที่ 1-31 นโยบายการเขียนข้อมูลแบบ Write-Through เพื่อความสอดคล้องของข้อมูล

- **ตัวอย่าง:** สมมติว่า CPU ต้องการเปลี่ยนค่าข้อมูลที่แอดเดรส X จาก 10 เป็น 20

  1. CPU พบข้อมูลแอดเดรส X ในแคช (Cache Hit) และทำการเปลี่ยนค่าในแคชจาก 10 เป็น 20
  2. **ทันทีที่ค่าในแคชเปลี่ยนเป็น 20** ระบบจะส่งค่า 20 นี้ไปเขียนทับที่แอดเดรส X ในหน่วยความจำหลักด้วย
  3. ผลลัพธ์: ทั้งในแคชและหน่วยความจำหลัก ข้อมูลที่แอดเดรส X จะเป็น 20 ทั้งคู่

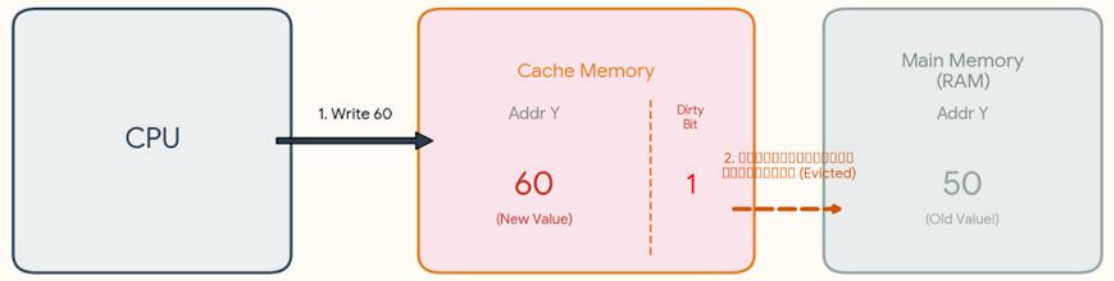
2. Write Back (เขียนกลับ)

- **แนวคิด:** เมื่อ CPU ทำการแก้ไขข้อมูลในแคช ข้อมูลนั้นจะถูกเขียนลงในแคชเท่านั้น และจะมีการตั้งค่า **"Dirty Bit"** (บิตสถานะ "สกปรก") ไว้ที่บล็อกนั้น เพื่อระบุว่าข้อมูลในแคชบล็อกนี้ไม่ตรงกับข้อมูลในหน่วยความจำหลัก ข้อมูลจะถูกเขียนกลับไปยังหน่วยความจำหลักก็ต่อเมื่อบล็อกนั้นถูกเลือกให้ออกไปจากแคช (เช่น เมื่อมีบล็อกใหม่เข้ามาแทนที่) หรือเมื่อมีการส่งสัญญาณให้เขียนกลับเท่านั้น
- **การทำงานโดยละเอียด:**
  1. CPU เขียนข้อมูลใหม่ลงในบล็อกของแคช
  2. ระบบจะตั้งค่า **Dirty Bit** ของบล็อกนั้นให้เป็น '1' เพื่อบ่งบอกว่าข้อมูลในบล็อกแคชนี้ได้มีการเปลี่ยนแปลงไปจากเดิม และยังไม่ได้เขียนกลับไปยังหน่วยความจำหลัก
  3. เมื่อบล็อกนี้ (ที่มี Dirty Bit เป็น '1') ถูกเลือกให้ออกจากแคช (ถูก "ไล่ออก" เพื่อเปิดพื้นที่ให้บล็อกอื่น) **เฉพาะตอนนั้นเท่านั้น** ที่ข้อมูลล่าสุดจากแคชจะถูกเขียนกลับไปยังหน่วยความจำหลัก
- **ข้อดี:** **ความเร็วในการเขียนสูงกว่า:** เนื่องจากไม่ต้องเขียนข้อมูลลงหน่วยความจำหลักทุกครั้งที่มีการเปลี่ยนแปลง ทำให้การเขียนข้อมูลลงแคชทำได้เร็วขึ้นมาก และลดจำนวนครั้งในการเข้าถึงหน่วยความจำหลัก

  ลดการใช้แบนด์วิดท์ของบัส: เพราะข้อมูลจะถูกเขียนกลับไปยังหน่วยความจำหลักเมื่อจำเป็นเท่านั้น

- **ข้อเสีย: ความเสี่ยงข้อมูลสูญหาย:** หากเกิดระบบขัดข้อง (เช่น ไฟดับ) ก่อนที่ข้อมูลที่มี Dirty Bit เป็น '1' จะถูกเขียนกลับไปยังหน่วยความจำหลัก ข้อมูลที่แก้ไขในแคชนั้นอาจสูญหายไป และทำให้ข้อมูลในแคชกับหน่วยความจำหลักไม่สอดคล้องกัน

  ซับซ้อนในการจัดการ: ต้องมีกลไกในการจัดการ Dirty Bit และต้องมีนโยบายในการ "ไล่ออก" บล็อก (Replacement Algorithm) ซึ่งเพิ่มความซับซ้อนให้กับระบบ



รูปที่ 1-32 นโยบายการเขียนข้อมูลแบบ Write-Back และกลไก Dirty Bit

ตัวอย่าง: สมมติว่า CPU ต้องการเปลี่ยนค่าข้อมูลที่แอดเดรส Y จาก 50 เป็น 60

1. CPU พบข้อมูลแอดเดรส Y ในแคช (Cache Hit) และทำการเปลี่ยนค่าในแคชจาก 50 เป็น 60
2. **Dirty Bit ของบล็อกที่เก็บแอดเดรส Y ในแคชจะถูกตั้งเป็น '1'** (ขณะนี้ ข้อมูลที่ Y ใน RAM ยังคงเป็น 50)
3. ต่อมา มีบล็อกข้อมูลใหม่ Z ต้องการเข้ามาในแคช และแคชเต็มพอดี ระบบจึงเลือกที่จะนำบล็อกที่เก็บแอดเดรส Y ออกไป
4. **ก่อนที่จะนำบล็อก Y ออกไป** ระบบจะตรวจสอบ Dirty Bit พบว่าเป็น '1' จึงทำการเขียนค่า 60 จากแคชกลับไปทับที่แอดเดรส Y ในหน่วยความจำหลัก
5. จากนั้นจึงนำบล็อก Y ออก และนำบล็อก Z เข้ามาแทนที่

ชุดข้อสอบเพื่อทบทวนความเข้าใจบทที่ 1: โครงสร้างระบบคอมพิวเตอร์

ส่วนที่ 1: ข้อสอบแบบตัวเลือก

คำชี้แจง: โปรดเลือกคำตอบที่ถูกต้องที่สุดเพียงข้อเดียวสำหรับแต่ละคำถาม

1. องค์ประกอบพื้นฐานที่สำคัญที่สุดของระบบคอมพิวเตอร์ที่ทำหน้าที่ในการประมวลผลข้อมูลคือส่วนใด?

ก. หน่วยรับข้อมูลเข้า (Input Unit)

ข. หน่วยแสดงผลข้อมูล (Output Unit)

ค. หน่วยประมวลผลกลาง (Central Processing Unit - CPU)

ง. หน่วยความจำสำรอง (Secondary Memory)

เฉลย: ค.

2. หน่วยวัดความเร็วของสัญญาณนาฬิกาของคอมพิวเตอร์ที่แสดงถึงจำนวนรอบต่อวินาทีคือหน่วยใด?

ก. Byte ข. Bit

ค. Hertz ง. Meter

เฉลย: ค.

3. หาก CPU ตัวเก่าใช้เวลาประมวลผลโปรแกรม 40 วินาที และ CPU ตัวใหม่ประมวลผลโปรแกรมเดียวกันเสร็จใน 25 วินาที ประสิทธิภาพความเร็ว (Speed up) ของ CPU ตัวใหม่เทียบกับตัวเก่าเป็นเท่าใด?

ก. 0.625 เท่า ข. 1.6 เท่า

ค. 15 เท่า ง. 65 เท่า

เฉลย: ข.

4. คอมพิวเตอร์ยุคแรก (ยุคที่ 1) ใช้เทคโนโลยีหลักใดในการประมวลผล?

ก. ทรานซิสเตอร์ (Transistor) ข. วงจรรวม (Integrated Circuit - IC)

ค. หลอดสูญญากาศ (Vacuum Tube) ง. ไมโครโปรเซสเซอร์ (Microprocessor)

เฉลย: ค.

5. ส่วนประกอบใดในหน่วยประมวลผลกลาง (CPU) ที่ทำหน้าที่ในการดำเนินการทางคณิตศาสตร์และตรรกะ?

ก. หน่วยควบคุม (Control Unit)

ข. หน่วยคำนวณและตรรกะ (Arithmetic Logic Unit - ALU)

ค. รีจิสเตอร์ (Registers)

ง. หน่วยความจำแคช (Cache Memory)

เฉลย: ข.

6. ในวงรอบคำสั่ง (Instruction Cycle) ขั้นตอน "Fetch" มีจุดประสงค์หลักคืออะไร?

ก. การดำเนินการตามคำสั่งที่อ่านเข้ามา

ข. การถอดรหัสคำสั่งเพื่อระบุการทำงาน

ค. การดึงคำสั่งจากหน่วยความจำหลักมายัง CPU

ง. การบันทึกผลลัพธ์ของการประมวลผล

เฉลย: ค.

7. ส่วนประกอบรูปแบบคำสั่ง (Instruction Format) ระบุชนิดของการดำเนินการที่ CPU ต้องทำคือส่วนใด?

ก. โอเปอแรนด์ (Operand)

ข. โอเปอเรชั่นโค้ด (Operation Code - OpCode)

ค. แอดเดรส (Address)

ง. บัส (Bus)

เฉลย: ข.

8. หาก CPU ต้องการอ่านข้อมูลจากหน่วยความจำ ตำแหน่งของข้อมูลที่ต้องการจะถูกส่งผ่านบัสประเภทใด?

ก. Data Bus

ข. Control Bus

ค. Address Bus

ง. I/O Bus

เฉลย: ค.

9. อุปกรณ์ใดจัดเป็นหน่วยรับข้อมูลเข้า (Input Unit)?

ก. เครื่องพิมพ์ (Printer)

ข. จอภาพ (Monitor)

ค. แป้นพิมพ์ (Keyboard)

ง. ลำโพง (Speaker)

เฉลย: ค.

10. เหตุใดหน่วยความจำ (Memory Unit) จึงมีความสำคัญอย่างยิ่งต่อการทำงานของระบบคอมพิวเตอร์?

ก. เป็นที่เก็บโปรแกรมและข้อมูลที่ CPU กำลังประมวลผล

ข. ทำหน้าที่เป็นแหล่งพลังงานหลักของระบบ

ค. ควบคุมการไหลของข้อมูลระหว่าง CPU และอุปกรณ์ I/O

ง. ประมวลผลคำสั่งทางคณิตศาสตร์และตรรกะ

เฉลย: ก.

11. ข้อใดกล่าวถูกต้องเกี่ยวกับลำดับชั้นของหน่วยความจำ (Memory Hierarchy) จากเร็วที่สุดไปช้าที่สุด?

ก. Secondary Memory > Main Memory > Cache > Registers

ข. Registers > Cache > Main Memory > Secondary Memory

ค. Cache > Registers > Main Memory > Secondary Memory

ง. Main Memory > Secondary Memory > Cache > Registers

เฉลย: ข.

12. หน่วยความจำสำรองประเภทใดที่ใช้เลเซอร์ในการอ่านและเขียนข้อมูล และมีตัวอย่างเช่น CD, DVD, Blu-ray?

ก. Magnetic Memory ข. Flash Memory

ค. Optical Memory ง. Solid State Drive

เฉลย: ค.

13. แนวคิด Locality of Reference (Temporal Locality) หมายถึงการทำงานของหน่วยความจำแคช?

ก. ข้อมูลที่อยู่ใกล้เคียงกันมีแนวโน้มถูกเข้าถึงพร้อมกัน

ข. ข้อมูลที่เพิ่งถูกใช้งานไปมีแนวโน้มจะถูกใช้งานอีกครั้งในอนาคตอันใกล้

ค. ข้อมูลที่ถูกใช้งานบ่อยที่สุดควรถูกเก็บไว้ในแคช

ง. ข้อมูลที่ถูกใช้งานน้อยที่สุดควรถูกนำออกจากแคช

เฉลย: ข.

14. การเข้าถึงข้อมูลในแคชสำเร็จ (Cache Hit) 95 ครั้ง จากเข้าถึงทั้งหมด 100 ครั้ง Hit Ratio เป็นเท่าใด?

ก. 0.05 ข. 0.95 ค. 95 ง. 100

เฉลย: ข. 0.95

15. วิธีการเชื่อมโยงข้อมูลของหน่วยความจำแคช (Cache Mapping) แบบใดที่แต่ละบล็อกของหน่วยความจำหลักสามารถถูกจัดเก็บได้เพียงบรรทัดเดียวในแคชเท่านั้น?

ก. Fully Associative Mapping ข. Set Associative Mapping

ค. Direct Mapping ง. Random Mapping

เฉลย: ค.

16. ในการออกแบบหน่วยความจำแคชแบบ Direct Mapping หากขนาดของบล็อก (Block Size) คือ 16 ไบต์ และมีแอดเดรสทั้งหมด 32 บิต จำนวนบิตสำหรับ Offset คือกี่บิต?

ก. 2 บิต ข. 4 บิต ค. 5 บิต ง. 8 บิต

เฉลย: ข.

17. อัลกอริทึมการเปลี่ยนบล็อกในแคช (Replacement Algorithm) แบบใดที่เลือกบล็อกที่ไม่ได้ถูกใช้งานมานานที่สุดออกไป?

ก. FIFO (First-In First-Out)
ข. LFU (Least Frequently Used)
ค. Random
ง. LRU (Least Recently Used)

เฉลย: ง.

18. นโยบายการเขียนข้อมูล (Write Policy) แบบใดที่เมื่อมีการแก้ไขข้อมูลในแคช ข้อมูลจะถูกเขียนกลับไปยังหน่วยความจำหลักด้วยทันที แต่ทำให้การเขียนช้าลง?

ก. Write Back
ข. Write Through
ค. Write Allocate
ง. No Write Allocate

เฉลย: ข.

ส่วนที่ 2: ข้อสอบอัตนัย (Essay Questions)

คำชี้แจง: โปรดอธิบายในแต่ละหัวข้อต่อไปนี้อย่างละเอียดและชัดเจน

1. **จงอธิบายและเปรียบเทียบความแตกต่างที่สำคัญระหว่างหน่วยควบคุม (Control Unit) และหน่วยคำนวณและตรรกะ (Arithmetic Logic Unit - ALU) ใน CPU พร้อมยกตัวอย่างการทำงานที่ชัดเจนของแต่ละส่วน**
    - **<u>แนวทางการตอบ</u>:** ควรอธิบายหน้าที่หลักของแต่ละส่วนอย่างชัดเจน ระบุความแตกต่างในบทบาทและประเภทของการประมวลผลที่รับผิดชอบ และให้ตัวอย่างสถานการณ์การทำงานที่แสดงให้เห็นถึงบทบาทของแต่ละส่วน เช่น Control Unit ควบคุมการ Fetch คำสั่ง ส่วน ALU ทำการบวกเลข
2. **จงอธิบายแนวคิดของลำดับชั้นหน่วยความจำ (Memory Hierarchy) โดยให้ระบุและเปรียบเทียบคุณสมบัติหลัก 3 ด้าน (ความเร็ว, ความจุ, ราคา) ของหน่วยความจำแต่ละระดับ (Registers, Cache, Main Memory, Secondary Memory)**
    - **<u>แนวทางการตอบ</u>:** ควรอธิบายว่า Memory Hierarchy คืออะไร จากนั้นระบุหน่วยความจำแต่ละระดับ พร้อมเปรียบเทียบในตารางหรือรูปแบบที่ชัดเจนถึงความเร็ว (เร็วที่สุด/ช้าที่สุด), ความจุ (น้อยที่สุด/มากที่สุด) และราคาต่อบิต (แพงที่สุด/ถูกที่สุด) ของแต่ละระดับ
3. **จงอธิบายหลักการทำงานของหน่วยความจำแคช (Cache Memory) พร้อมทั้งอธิบายแนวคิดของ Locality of Reference ทั้งแบบ Spatial Locality และ Temporal Locality อย่างละเอียด และยกตัวอย่างที่แสดงให้เห็นถึงการประยุกต์ใช้แนวคิดเหล่านี้ใน Cache**

- **<u>แนวทางการตอบ</u>:** ควรอธิบายว่าแคชคืออะไร ทำไมถึงต้องมี จากนั้นอธิบาย Locality of Reference โดยรวม และแยกอธิบาย Spatial Locality (ข้อมูลที่อยู่ติดกันมักถูกใช้พร้อมกัน) และ Temporal Locality (ข้อมูลที่เพิ่งใช้มักถูกใช้ซ้ำ) พร้อมยกตัวอย่างโปรแกรมที่แสดงให้เห็นแนวคิดเหล่านี้ (เช่น การวนลูป, การเข้าถึงอาร์เรย์)

4. **จงอธิบายและเปรียบเทียบวิธีการเชื่อมโยงข้อมูลของหน่วยความจำแคช (Cache Mapping) ทั้ง 3 แบบ (Direct Mapping, Fully Associative Mapping, และ Set Associative Mapping) โดยเน้นความแตกต่างในวิธีการระบุตำแหน่งบล็อกข้อมูลในแคชและข้อดีข้อเสียของแต่ละวิธี**
   - **<u>แนวทางการตอบ</u>:** ควรอธิบายแต่ละวิธีโดยเน้นว่าแอดเดรสถูกใช้ในการระบุตำแหน่งในแคชอย่างไร (เช่น Direct ใช้ Index, Fully Associative ไม่มี Index แต่ใช้ Tag ทั้งหมด, Set Associative ใช้ Index ระบุ Set) พร้อมระบุข้อดีข้อเสียของแต่ละวิธี (เช่น Direct ง่ายแต่เกิด Conflict Miss บ่อย, Fully Associative ยืดหยุ่นแต่ซับซ้อนและแพง, Set Associative เป็นทางสายกลาง)
5. **จงอธิบายแนวคิดและขั้นตอนการทำงานของวงรอบคำสั่ง (Instruction Cycle) ทั้งในส่วนของ Fetch และ Execute ให้ชัดเจน พร้อมระบุบทบาทของรีจิสเตอร์ที่เกี่ยวข้อง (เช่น Program Counter - PC, Instruction Register - IR, Memory Address Register - MAR, Memory Buffer Register - MBR)**
   - **<u>แนวทางการตอบ</u>:** ควรเริ่มจากการอธิบายว่า Instruction Cycle คืออะไร จากนั้นอธิบายขั้นตอน Fetch (การดึงคำสั่ง) และ Execute (การประมวลผลคำสั่ง) โดยละเอียดในแต่ละขั้น พร้อมระบุว่ารีจิสเตอร์แต่ละตัวมีบทบาทอย่างไรในแต่ละขั้นตอน (เช่น PC ชี้คำสั่งถัดไป, IR เก็บคำสั่งที่กำลังประมวลผล, MAR/MBR ใช้ในการติดต่อหน่วยความจำ)